

DOCUMENTAZIONE TECNICA

Middleware MyPay — Guida Tecnica Completa

Versione: 5.0.0

Data: 05 Maggio 2026

Stato: Middleware completo con 40 operazioni SOAP su 10 endpoint, proxy upload flusso import, routing dinamico PU/LEGACY per-ente

Questo documento è la Single Source of Truth (SSoT) del progetto `mypay.mypaycore`. Tutti gli agenti OpenCode (`.opencode/agents/*.md`) e il file `AGENTS.md` fanno riferimento a questo documento per il contesto tecnico completo. Evitare di duplicare informazioni di progetto nei file degli agenti.

Indice

1. [Cos'è questo progetto](#)
2. [Architettura generale](#)
3. [Struttura del progetto Maven](#)
4. [Struttura dei pacchetti Java](#)
5. [Modulo di Configurazione](#)
6. [Modulo di Autenticazione](#)
7. [Modulo Client Piattaforma](#)
8. [Modulo Persistenza \(domain + repository\)](#)
9. [Modulo Routing](#)
10. [Modulo Endpoint SOAP](#)
11. [Proxy Upload Flusso Import](#)
12. [Gestione degli Errori](#)
13. [Monitoraggio e Health Check](#)
14. [Resilienza](#)
15. [Ambienti e Profili](#)
16. [Test](#)
17. [Come avviare il progetto](#)
18. [Come testare il flusso completo](#)
19. [Cosa NON è ancora implementato](#)
20. [Prossimi passi — Fasi Future](#)
21. [Agenti OpenCode](#)

1. Cos'è questo progetto

`mypay.mypaycore` è un **middleware di integrazione** sviluppato con il framework **SpringLine2** di ARIA S.p.A. (Azienda Regionale per l'Innovazione e gli Acquisti, Regione Lombardia).

Problema che risolve

I **SIL** (Sistemi Informativi Locali) degli enti pubblici (comuni, province, ecc.) devono comunicare con la **Piattaforma Unitaria** di pagoPA per gestire pagamenti, riconciliazioni e flussi di tesoreria. La Piattaforma Unitaria richiede autenticazione OAuth2, che i SIL non sono in grado di gestire autonomamente.

Il middleware si interpone tra i due sistemi e:

- Espone **49 operazioni SOAP** distribuite su **10 endpoint** ai SIL (9 MyPay + 1 MyPivot)
- Gestisce in autonomia l'**autenticazione OAuth2** verso pagoPA
- Identifica l'ente tramite **ricerca generica** nel payload SOAP (`<codIpaEnte>` o `<identificativoDominio>`)
- Supporta **routing dinamico** per ente (DB-driven) con **cache duale** (codIpa + codiceFiscale)
- **Inoltra** le richieste autenticate alla Piattaforma Unitaria
- **Restituisce** le risposte ai SIL nel formato atteso

Flusso ad alto livello



NOTA IMPORTANTE: Il SIL **NON** invia un JWT/Bearer token. L'autenticazione del SIL avviene tramite `codIpaEnte` (nell'Header SOAP) e `password` (nel Body SOAP). Il middleware gestisce internamente l'autenticazione OAuth2 verso la Piattaforma Unitaria. Il routing PU vs LEGACY è implementato nel `RoutingDecisionService`. Ogni richiesta viene instradata dinamicamente in base alla **presenza/assenza** di una configurazione attiva per l'ente nella tabella `mygov_ente_config_pu`. Le credenziali OAuth2 sono **per-ente**: ogni ente ha il proprio `client_id` e `client_secret`.

2. Architettura generale

Framework base: SpringLine2

Il progetto è costruito su **SpringLine2** (versione 2027.01.01), un framework proprietario ARIA che estende Spring Boot 3.5.5 e fornisce:

Componente SpringLine2	Funzione
springline2-core	Nucleo: sicurezza, logging MON/APP, configurazione cloud
springline2-ws	Client SOAP (lato client verso sistemi esterni)

springline2-openapi	Integrazione Swagger/OpenAPI automatica
---------------------	---

Dipendenze aggiuntive (non SpringLine2)

Dipendenza	Versione	Scopo
spring-boot-starter-web-services	gestita da Spring Boot	Server SOAP (riceve richieste dai SIL)
spring-boot-starter-web	gestita da Spring Boot	Controller REST (@RestController) — aggiunto per il Proxy Upload Flusso Import
jakarta.xml.bind-api	gestita da Spring Boot	Marshalling/unmarshalling XML
jaxb-runtime	gestita da Spring Boot	Implementazione runtime JAXB
resilience4j-spring-boot3	2.2.0	Circuit Breaker e Retry
spring-boot-starter-aop	gestita da Spring Boot	Necessario per le annotazioni Resilience4j
spring-boot-starter-actuator	gestita da Spring Boot	Health check e metriche
spring-boot-starter-jdbc	gestita da Spring Boot	Supporto JDBC e transaction manager Spring
jdbi3-spring5	3.27.0	Integrazione Jdbi con contesto Spring
jdbi3-sqlobject	3.27.0	DAO dichiarativi basati su interfacce e annotazioni SQL
jdbi3-stringtemplate4	3.27.0	Template SQL dinamici per query Jdbi
postgresql	gestita da Spring Boot	Driver JDBC PostgreSQL

Moduli Maven del progetto

mypay.mypaycore/	← POM padre (aggregator)
└ mypay.mypaycore-springboot/	← Applicazione Spring Boot (questo è il cuore)
└ mypay.mypaycore-db/	← Script SQL PostgreSQL (6 script: tabelle, alter, dati esempio)
└ mypay.mypaycore-properties/	← Modulo proprietà (SpringLine2 config)
└ mypay.mypaycore-release/	← Modulo di packaging per il rilascio

3. Struttura del progetto Maven

mypay.mypaycore-springboot (modulo principale)

Contiene tutto il codice sorgente Java dell'applicazione.

mypay.mypaycore-springboot/
└ pom.xml
└ src/
└ main/
└ java/it/ariaspa/mypay/mypaycore/api/
└ Application.java

```

|   ├── auth/
|   ├── client/
|   ├── common/
|   ├── config/
|   ├── domain/
|   ├── health/
|   ├── logging/
|   ├── metrics/
|   ├── repository/
|   ├── routing/
|   ├── soap/
|   └── util/
└── resources/config/
    ├── application.properties      ← configurazione base (comune a tutti i profili)
    ├── application-dev.properties  ← profilo sviluppo (unico profilo attivo)
    └── bootstrap.properties

```

mypay.mypaycore-db (modulo database)

Contiene gli script SQL PostgreSQL per la creazione delle tabelle del middleware:

Script	Scopo
000_PLACEHOLDER.sql	Placeholder vuoto
002_CREATE_MWPAY_TRANSACTION_LOG.sql	Tabella log transazionale delle richieste SOAP (include modalità SCONOSCIUTA)
004_CREATE_MYGOV_ENTE_CONFIG_PU.sql	Nuova tabella configurazione OAuth2 per-ente (post refactoring multi-ente)
005_DROP_MWPAY_ENTE_CONFIG.sql	Rimozione vecchia tabella monolitica (eseguire in fase di migrazione)
006_INSERT_ENTE_CONFIG_PU_EXAMPLE.sql	Dati di esempio per l'ambiente di sviluppo
007_ALTER_MYGOV_ENTE_CONFIG_PU.sql	Allinea lo schema della tabella al modello Java EnteConfigPu (rinomina colonne, aggiunge attivo, dt_creazione, dt_ultima_modifica, indice composito, vincolo UNIQUE — Fase 8)

Il modulo è gestito dal plugin `custom-package-plugin` (ARIA) che produce un archivio ZIP con gli script per il deployment.

Nota: Il tag `<summary>` è stato rimosso dalla configurazione del plugin perché non riconosciuto dalla versione `3.2.0` (causava un falso positivo in IntelliJ pur non compromettendo la build).

mypay.mypaycore-properties (modulo di configurazione per il deployment)

Contiene i file di configurazione utilizzati durante il deployment su server. I file sono in formato `.properties` :

```

mypay.mypaycore-properties/
└── src/main/resources/
    ├── application.properties      ← template di configurazione per il deployment (DataSource, SSL, SpringLine2)
    ├── bootstrap.properties        ← versione applicazione e configurazioni di bootstrap
    ├── logback-spring.xml          ← configurazione Logback per console e file di log
    └── startup.sh                  ← script di avvio con --spring.profiles.active=dev

```

Il file `application.properties` in questo modulo è il **template di deployment**: contiene placeholder (`<INSERIRE ...>`) per tutte le proprietà sensibili (credenziali DB, password SSL, segreto JWT). Va completato prima del deploy su ogni ambiente.

Il file `logback-spring.xml` definisce la strategia di scrittura dei log su console e su file, separando i flussi tecnici generali da quelli di monitoraggio, audit e tracing.

logback-spring.xml

Il file `mypay.mypaycore-properties/src/main/resources/logback-spring.xml` configura gli appender Logback del progetto e usa la proprietà `logging.file.dir` come directory base per i file prodotti.

File di log configurati:

File	Contenuto previsto
<code>backend.log</code>	Log tecnici generali applicativi e di infrastruttura
<code>mon.log</code>	Log di monitoraggio in formato compatto
<code>app.log</code>	Log applicativi dedicati in formato compatto
<code>audit.log</code>	Eventi di audit in formato solo messaggio
<code>filter.log</code>	Trace di filtri web e message tracing SOAP/client

Dettagli operativi:

- usa `RollingFileAppender` con rotazione giornaliera per tutti i file principali
- mantiene separati i flussi per semplificare troubleshooting, audit e monitoraggio operativo
- lascia il root logger attivo su console e su `backend.log`
- consente di instradare logger specifici su file dedicati in base al package o al ruolo funzionale

4. Struttura dei pacchetti Java

Tutti i sorgenti si trovano sotto: `it.ariaspa.mypay.mypaycore.api`

```
api/
├─ Application.java                ← Entry point Spring Boot (@EnableScheduling)
├─
├─ config/                        ← Configurazione applicazione
│   ├── PiattaformaUnitariaConfig.java    (OAuth2 e URL Piattaforma Unitaria)
│   ├── SoapWebServiceConfig.java         (@EnableWs + servlet su /ws/*)
│   ├── PathRegistryConfig.java           (mapping path-prefix → backend – Fase 5)
│   ├── BackendRoutingConfig.java         (URL backend mypay/mypivot – Fase 5)
│   ├── DataSourceConfiguration.java      (DataSource PostgreSQL con HikariCP)
│   └── JdbiConfiguration.java            (istanza Jdbi primaria)
├─
├─ auth/                          ← Autenticazione OAuth2
│   ├── OAuthTokenService.java           (multi-ente: ConcurrentHashMap<codIpaEnte, TokenData>)
│   └── dto/
│       └── OAuthTokenResponse.java
├─
├─ routing/                       ← Logica di routing (Fase 7)
│   ├── RoutingDecision.java             (risultato immutabile: destinazione, modalita, urlBackend, ente)
│   └── RoutingDecisionService.java       (cervello del routing: path + DB → decisione)
├─
├─ client/                        ← Client HTTP
│   ├── PiattaformaUnitariaClient.java   (verso PU con OAuth2 per-ente – nessun interceptor)
│   ├── ProxyForwardingClient.java       (forward legacy senza OAuth2 – Fase 5)
│   └── UploadForwardingClient.java       (upload file verso backend legacy o PU – Proxy Upload)
├─
├─ upload/                        ← Proxy upload flusso import
│   └── UploadProxyEntry.java             (DTO immutabile per la cache: uploadUrl, modalità, ente...)
```

- | | UploadProxyCacheService.java (cache TTL in-memory one-shot con pulizia @Scheduled)
- | | UpdateFilePuService.java (validazione versione tracciato file per upload verso PU)
- | | UploadFlussoController.java (@RestController – POST /api/upload/flusso)
- |
- | soap/ ← Endpoint SOAP lato SIL
- | | endpoint/
- | | | AbstractSoapProxyEndpoint.java (classe base: proxy trasparente con identificazione ente duale, getFaultDetailNamespace() astratto, intercettazione SOAP Fault backend)
- | | | mypay/ ← 4 endpoint MyPay PA (23 operazioni)
- | | | | PagamentiTelematiciDovutiPagatiEndpoint.java (16 operazioni – con post-processing paaSILAutorizzaImportFlusso)
- | | | | PagamentiTelematiciCCPPaEndpoint.java (4 operazioni)
- | | | | PagamentiTelematiciEsitoEndpoint.java (1 operazione)
- | | | | PagamentiTelematiciFlussiSPCEndpoint.java (2 operazioni)
- | | | mypay/fesp/ ← 5 endpoint MyPay FESP (16 operazioni)
- | | | | PagamentiTelematiciRPEndpoint.java (8 operazioni)
- | | | | PagamentiTelematiciCCP25Endpoint.java (5 operazioni)
- | | | | PagamentiTelematiciCCPEndpoint.java (2 operazioni)
- | | | | PagamentiTelematiciRTEndpoint.java (1 operazione)
- | | | | PagamentiTelematiciAvvisiDigitaliEndpoint.java (1 operazione)
- | | | mypivot/ ← 1 endpoint MyPivot (10 operazioni, con post-processing pivotSILAutorizzaImportFlusso)
- | | | | ReconciliationEndpoint.java (10 operazioni – post-processing su pivotSILAutorizzaImportFlusso)
- | | exception/
- | | | SoapFaultExceptionResolver.java (7 tipi di eccezione mappati, Ordered.HIGHEST_PRECEDENCE, namespace dinamico)
- | | | FaultCode.java (enum centralizzato dei codici fault SOAP)
- |
- | domain/ ← Modelli dati
- | | ModalitaRouting.java (enum: PIATTAFORMA_UNITARIA, LEGACY)
- | | Ente.java (modello tabella mygov_ente – DB condiviso)
- | | EnteConfigPu.java (modello tabella mygov_ente_config_pu)
- | | EnteCompleto.java (aggregato Ente + EnteConfigPu)
- | | TransactionLog.java (modello tabella mygov_mw_transaction_log)
- |
- | repository/ ← Accesso dati Jdbi
- | | EnteRepository.java (DAO Jdbi per mygov_ente)
- | | EnteRowMapper.java
- | | EnteConfigPuRepository.java (DAO Jdbi per mygov_ente_config_pu)
- | | EnteConfigPuRowMapper.java
- | | EnteCacheService.java (cache TTL in-memory – ConcurrentHashMap<codIpaEnte, EnteCompleto>)
- | | TransactionLogRepository.java (DAO Jdbi – SqlObject)
- | | TransactionLogRowMapper.java
- |
- | logging/ ← Log transazionale e marker (Fase 9)
- | | TransactionLoggingService.java (log su DB con resilienza – mai blocca il SIL)
- | | JdbiSqlLogger.java (logger SQL custom per Jdbi)
- | | LogMarker.java (marker SLF4J centralizzati)
- |
- | metrics/ ← Metriche Micrometer (Fase 9)
- | | MiddlewareMetricsService.java (Counter, Timer, Gauge per Actuator)
- |
- | common/ ← Classi condivise
- | | exception/
- | | | PiattaformaAuthenticationException.java

```

|      |─ PiattaformaCommunicationException.java
|      |─ EnteNonCensitoException.java      (Fase 7 – aggiornata: senza tipoOperazione)
|      |─ EnteNonIdentificabileException.java (31 Mar 2026 – ente presente ma non identificabile dall'header
SOAP)
|      |─ PathNonRiconosciutoException.java (Fase 7)
|
|─ health/                                ← Health check Actuator
|  |─ OAuthTokenHealthIndicator.java      (itera su tutti gli enti in cache token)
|  |─ PiattaformaUnitariaHealthIndicator.java
|  |─ EnteConfigHealthIndicator.java      (Fase 9 – usa EnteCacheService)
|
|─ util/                                  ← Utility tecniche condivise
|  |─ Constants.java                      (contenitore centralizzato delle costanti, incluse NS_FAULT_MYPAY e
NS_FAULT_MYPIVOT)
|  |─ LogHelper.java                      (utility per firme metodo leggibili nei log)
|  |─ SupportedFileVersion.java           (enum versioni file riconosciute per la validazione upload PU)
|  |─ Utilities.java                      (helper tecnici riusabili)

```

5. Modulo di Configurazione

PiattaformaUnitariaConfig.java

Tipo: `@Configuration` + `@ConfigurationProperties(prefix = "piattaforma-unitaria")`

Scopo: Centralizza tutti i parametri di connessione verso la Piattaforma Unitaria.

Questa classe legge automaticamente dal file `application.properties` il blocco:

```

piattaforma-unitaria.base-url=https://api.uat.p4pa.pagopa.it
piattaforma-unitaria.auth.token-url=${piattaforma-unitaria.base-url}/pu/auth/oauth/token
piattaforma-unitaria.auth.grant-type=client_credentials
piattaforma-unitaria.auth.scope=openid

```

Proprietà esposte:

Proprietà	Tipo	Descrizione
baseUr1	String	URL base della Piattaforma Unitaria
auth.tokenUr1	String	Endpoint OAuth2 per il token
auth.grantType	String	Sempre <code>client_credentials</code>
auth.scope	String	Sempre <code>openid</code>

SoapWebServiceConfig.java

Tipo: `@Configuration` + `@EnableWs` + `implements WsConfigurer`

Scopo: Configura Spring WS per esporre endpoint SOAP in ricezione dai SIL.

Cosa fa:

- Registra un `MessageDispatcherServlet` mappato su `/ws/*`
- Questo servlet intercetta tutte le richieste HTTP POST con Content-Type `text/xml` verso i path dei backend (`/ws/pivot/*` , `/ws/pa/*` , `/ws/fesp/*`)
- I singoli endpoint SOAP (annotati con `@Endpoint`) vengono rilevati automaticamente

Nota tecnica importante: `springline2-ws` è una libreria client SOAP (per chiamare servizi esterni). Per esporre endpoint SOAP server-side è necessaria la dipendenza separata `spring-boot-starter-web-services`, che è stata aggiunta esplicitamente al pom.

PathRegistryConfig.java (Fase 5)

Tipo: `@Configuration` + `@ConfigurationProperties(prefix = "routing")`

Scopo: Registro configurabile dei path-prefix e il loro mapping verso i backend (mypay o mypivot).

Funzionamento:

- Legge la mappa `routing.path-map.*` da `application.properties`
- Converte le chiavi normalizzate in path reali (`ws-pivot` → `/ws/pivot`)
- Metodo `resolveBackend(String requestPath)` → `Optional<BackendDestinatario>` con longest-prefix matching
- Enum interno `BackendDestinatario` (`MYPAY`, `MYPIVOT`)
- Validazione `@PostConstruct` : fallisce se il path-map è vuoto

Proprietà:

```
routing.path-map.ws-pivot=MYPIVOT
routing.path-map.ws-pa=MYPAY
routing.path-map.ws-fesp=MYPAY
```

BackendRoutingConfig.java (Fase 5)

Tipo: `@Configuration` + `@ConfigurationProperties(prefix = "backend")`

Scopo: Centralizza gli URL dei backend legacy (mypay e mypivot).

Proprietà:

```
backend.mypivot.base-url=${BACKEND_MYPIVOT_URL:http://localhost:8081}
backend.mypay.base-url=${BACKEND_MYPAY_URL:http://localhost:8082}
```

Metodo pubblico:

- `getBaseUrlFor(BackendDestinatario backend)` → `String` — restituisce l'URL base in base al tipo di backend

DataSourceConfiguration.java

Tipo: `@Configuration`

Scopo: Configura il DataSource PostgreSQL `pa` (database principale di MyPay) con pool HikariCP per l'accesso JDBC/Jdbi.

Perché è necessaria: Spring Boot auto-configura il datasource solo se le proprietà seguono il prefisso standard `spring.datasource.*`. Il progetto usa il prefisso personalizzato `spring.datasource.pa.*` (convenzione ereditata dal progetto mypay4 legacy), quindi è necessaria una classe di configurazione esplicita.

Bean creati:

Bean	Tipo	Descrizione
<code>paDataSourceProperties</code>	<code>DataSourceProperties</code>	Legge <code>spring.datasource.pa.*</code> (url, username, password, driver)
<code>dsPa</code>	<code>DataSource/HikariDataSource</code>	Pool di connessioni; parametri da <code>spring.datasource.pa.hikari.*</code>
<code>tmPa</code>	<code>DataSourceTransactionManager</code>	Gestione transazioni JDBC condivisa con Jdbi

Tutti i bean sono annotati con `@Primary` poiché è il datasource unico dell'applicazione.

Dettagli rilevanti:

- supporta password cifrata tramite `spring.datasource.cryptPassword=true` e decrypt Jasyp
- forza `autoCommit=false` sul datasource per demandare il controllo transazionale a Spring
- espone il datasource con nome `dsPa`, riutilizzato dalla configurazione Jdbi

Configurazione Properties corrispondente (esempio profilo `dev`):

```
spring.datasource.pa.driver-class-name=org.postgresql.Driver
spring.datasource.pa.url=${DB_PA_URL:jdbc:postgresql://localhost:5432/mypay_local_copy}
spring.datasource.pa.username=${DB_PA_USERNAME:admin}
spring.datasource.pa.password=${DB_PA_PASSWORD:admin}
spring.datasource.pa.hikari.minimum-idle=1
spring.datasource.pa.hikari.maximum-pool-size=5
spring.datasource.pa.hikari.pool-name=HikariPool-PA-dev
```

JdbiConfiguration.java

Tipo: `@Configuration`

Scopo: Completa il layer di persistenza configurando l'istanza Jdbi primaria collegata al datasource `dsPa`.

Cosa fa:

- crea il bean `jdbiPa` usato dai componenti di accesso dati
- avvolge il datasource in `TransactionAwareDataSourceProxy` per partecipare correttamente alle transazioni Spring
- applica timeout globali alle query tramite `SqlStatements`
- abilita il logging SQL tramite `JdbiSqlLogger` quando richiesto dalle proprietà
- installa automaticamente plugin Jdbi e `RowMapper` presenti nel contesto Spring
- registra `SqlObjectPlugin` per supportare DAO dichiarativi con annotazioni `@SqlQuery`, `@SqlUpdate`, ecc.

Bean creati:

Bean	Tipo	Descrizione
<code>jdbiPa</code>	<code>Jdbi</code>	Istanza primaria di Jdbi associata al datasource <code>dsPa</code>
<code>sqlObjectPlugin</code>	<code>JdbiPlugin</code>	Abilita il modello SQL Object per DAO/interfacce annotate
<code>messageSource</code>	<code>ResourceBundleMessageSource</code>	Message source condiviso per messaggi applicativi

Utilizzo consigliato:

- definire DAO o repository Jdbi invece di entity e repository JPA
- usare `@Transactional` sui servizi Spring quando più operazioni Jdbi devono partecipare alla stessa transazione
- registrare `RowMapper` dedicati per il mapping dei result set verso DTO o model applicativi

6. Modulo di Autenticazione

OAuthTokenResponse.java (DTO)

Tipo: POJO con annotazioni Jackson e Lombok

Scopo: Deserializza la risposta JSON dell'endpoint OAuth2 in un oggetto Java.

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "Bearer",
```

```
"expires_in": 3600
}
```

Note di sicurezza: Il campo `accessToken` è escluso dal metodo `toString()` con `@ToString(exclude = "accessToken")` per evitare che il token appaia accidentalmente nei log.

OAuthTokenService.java

Tipo: `@Service`

Scopo: Gestisce il ciclo di vita completo dei token OAuth2 in modalità **multi-ente**. Ogni ente ha le proprie credenziali (`clientId` e `clientSecret`) memorizzate in `mygov_ente_config_pu` ; i token vengono mantenuti in una cache separata per ente.

Struttura interna:

```
ConcurrentHashMap<codIpaEnte, TokenData>
    dove TokenData contiene: accessToken, tokenExpiryTime
```

Funzionamento dettagliato:

```
getAccessToken(codIpaEnte, clientId, clientSecret) chiamato
|
▼
Token valido in cache per questo ente?
(tokenData != null
  && Instant.now() < tokenExpiryTime)
|
Sì —→ restituisce accessToken dalla cache (nessuna chiamata HTTP)
|
No —→ acquisisce lock per questo ente
      (solo un thread alla volta richiede il token per lo stesso ente)
|
▼
Double-check: token ancora non valido?
|
Sì —→ POST token-url con client_credentials (clientId + clientSecret dell'ente)
      I parametri OAuth2 vengono inviati come **query string** nell'URL:
      POST token-url?client_id=...&client_secret=...&grant_type=client_credentials&scope=openid
      NOTA: la PU restituisce 404 se i parametri vengono inviati come body form-urlencoded
|
▼
Salva token + calcola scadenza
tokenExpiryTime = now + expires_in - 60 secondi (margine sicurezza)
|
▼
rilascia lock —→ restituisce token
```

Parametri tecnici:

- Connect timeout verso OAuth2: **5 secondi**
- Read timeout verso OAuth2: **10 secondi**
- Margine di sicurezza prima della scadenza: **60 secondi**
- Thread-safety: garantita tramite lock per ente + struttura `ConcurrentHashMap`

Metodi pubblici:

Metodo	Descrizione
--------	-------------

<code>getAccessToken(codIpaEnte, clientId, clientSecret)</code>	Restituisce token valido (dalla cache per-ente o nuovo)
<code>refreshToken(codIpaEnte, clientId, clientSecret)</code>	Forza il refresh del token per un ente specifico (usato dopo 401)
<code>isTokenValid(codIpaEnte)</code>	Verifica se il token di un ente è ancora valido
<code>getTokenCacheSize()</code>	Numero di enti con token in cache — usato dall'health indicator
<code>getEntiInCache()</code>	Set dei codici IPA degli enti con token in cache

7. Modulo Client Piattaforma

PiattaformaUnitariaClient.java

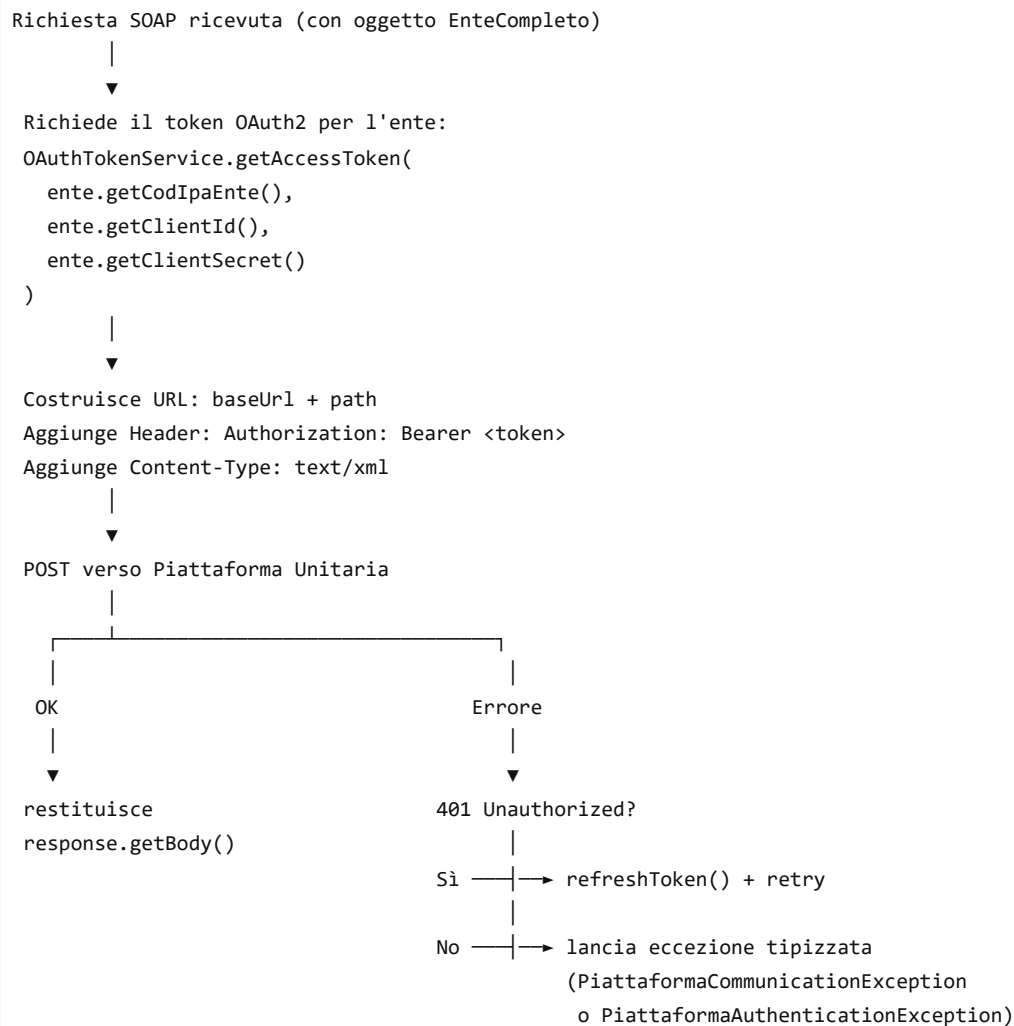
Tipo: `@Service`

Scopo: Unico punto di accesso al sistema esterno (Piattaforma Unitaria pagoPA).

Configurazione HTTP:

- Connect timeout: **5 secondi**
- Read timeout: **30 secondi** (le chiamate SOAP possono essere lente)
- **Nessun interceptor** — il token Bearer viene aggiunto manualmente nel metodo (refactoring multi-ente)

Flusso di `forwardSoapRequest(path, soapXml, ente)` (firma aggiornata nel refactoring multi-ente):



Resilienza applicata (vedi sezione 11):

- `@CircuitBreaker(name = "piattaformaUnitaria")` : protegge da cascate di errori
- `@Retry(name = "piattaformaUnitaria")` : tentativi con backoff esponenziale
- `forwardSoapRequestFallback()` : risposta di fallback quando il circuit breaker è aperto

ProxyForwardingClient.java (Fase 5)

Tipo: `@Service`

Scopo: Client HTTP per il forward trasparente verso i backend legacy (mypay/mypivot) in modalità legacy.

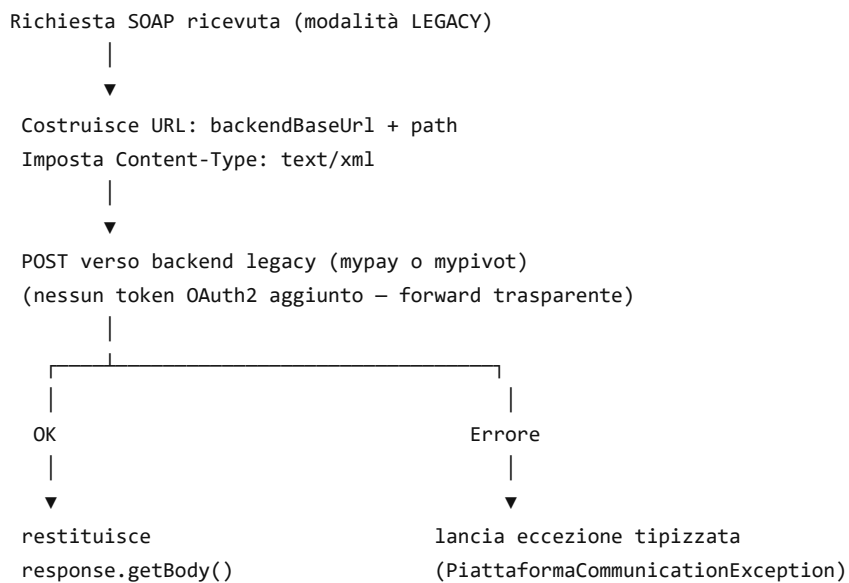
Differenze rispetto a `PiattaformaUnitariaClient` :

- **Nessun token OAuth2** — le credenziali SIL (`codIpaEnte` + `password`) viaggiano as-is nel payload SOAP
- `RestTemplate` separato, senza `OAuthTokenInterceptor`
- Nessuna trasformazione del payload
- Circuit breaker e retry dedicati (`backendLegacy`)

Configurazione HTTP:

- Connect timeout: **5 secondi**
- Read timeout: **30 secondi**
- Content-Type: `text/xml` (SOAP)

Flusso di `forwardToBackend(String backendBaseUrl, String path, String soapXml)` :



Resilienza applicata:

- `@CircuitBreaker(name = "backendLegacy")` : circuit breaker dedicato (finestra 10, soglia 50%, attesa 30s)
- `@Retry(name = "backendLegacy")` : retry con backoff esponenziale (3 tentativi, 1s → 2s → 4s)
- `forwardToBackendFallback()` : risposta di fallback quando il circuit breaker è aperto

8. Modulo Persistenza (domain + repository)

Questo modulo gestisce il layer di persistenza del middleware: modelli di dominio, accesso ai dati tramite Jdbi e cache in-memory degli enti.

Modelli di dominio (`domain/`)

ModalitaRouting.java (enum)

Tipo: `enum`

Scopo: Rappresenta le due modalità di instradamento di una richiesta SOAP.

Valore	Descrizione
PIATTAFORMA_UNITARIA	Inoltro con autenticazione OAuth2 verso la PU di pagoPA — il middleware aggiunge automaticamente il token Bearer
LEGACY	Forward diretto al backend legacy (mypay o mypivot) — le credenziali SIL (codIpaEnte + password) viaggiano as-is nel body SOAP

Ente.java (modello)

Tipo: POJO

Scopo: Rappresenta un ente pubblico nella tabella `mygov_ente`, che è una tabella **esistente nel DB condiviso** con mypay/mypivot (non creata dal middleware).

Campi principali:

Campo	Tipo	Descrizione
<code>mygovEnteId</code>	Long	Chiave surrogata auto-generata
<code>codIpaEnte</code>	String	Codice IPA dell'ente (chiave business univoca, es. "R_LOMBARDIA")
<code>codiceFiscaleEnte</code>	String	Codice fiscale dell'ente (chiave alternativa — usato come <code>identificativoDominio</code> nelle richieste SOAP FESP)
<code>deNomeEnte</code>	String	Denominazione estesa dell'ente
<code>cdStatoEnte</code>	String	Codice di stato dell'ente (non usato per la validazione del routing)

EnteConfigPu.java (modello)

Tipo: POJO

Scopo: Rappresenta la configurazione OAuth2 per-ente nella tabella `mygov_ente_config_pu`. Ogni ente può avere al massimo una riga (relazione 1:1 con `mygov_ente`).

Campi:

Campo	Tipo	Descrizione
<code>codIpaEnte</code>	String	Codice IPA — FK verso <code>mygov_ente.cod_ipa_ente</code>
<code>clientId</code>	String	Client ID OAuth2 specifico per questo ente
<code>clientSecret</code>	String	Client Secret OAuth2 specifico per questo ente (cifrato in produzione)
<code>attivo</code>	boolean	Flag di attivazione — se <code>false</code> , l'ente viene trattato come LEGACY
<code>dtCreazione</code>	LocalDateTime	Data e ora di creazione del record
<code>dtUltimaModifica</code>	LocalDateTime	Data e ora dell'ultimo aggiornamento

Naming convention: prefisso `mygov_` (non `mwpay_`) perché questa tabella è parte del dominio PA e si allinea con le convenzioni del DB condiviso.

EnteCompleto.java (aggregato)

Tipo: POJO (classe aggregato)

Scopo: Combina `Ente` + `EnteConfigPu` in un unico oggetto che rappresenta lo stato completo di un ente. Questo è il tipo restituito da `EnteCacheService`.

Metodi di convenienza:

Metodo	Descrizione
isPiattaformaUnitaria()	true se enteConfigPu != null && enteConfigPu.isAttivo() — l'ente è abilitato per la PU
getClientId()	Delega a enteConfigPu.getClientId()
getClientSecret()	Delega a enteConfigPu.getClientSecret()
getCodIpaEnte()	Delega a ente.getCodIpaEnte()

Logica di routing derivata: il routing non è più basato su una colonna `modalita_routing` esplicita, ma sulla **presenza/assenza** della configurazione PU:

- `EnteCompleto.isPiattaformaUnitaria() == true` → instrada verso la Piattaforma Unitaria con OAuth2
- `EnteCompleto.isPiattaformaUnitaria() == false` → instrada verso il backend legacy

TransactionLog.java (modello)

Tipo: POJO

Scopo: Rappresenta il log di una singola transazione SOAP processata dal middleware. Corrisponde a un record della tabella `mygov_mw_transaction_log`.

Campi (11):

Campo	Tipo	Descrizione
id	Long	Identificativo univoco (chiave surrogata auto-generata)
codIpaEnte	String	Codice IPA dell'ente che ha effettuato la richiesta
tipoOperazione	String	Tipo di operazione SOAP — internamente usa costante "N/A" dopo il refactoring
modalitaRouting	ModalitaRouting	Modalità di instradamento utilizzata (PU o legacy)
destinazione	String	Backend di destinazione: "MYPAY" o "MYPIVOT"
pathRichiesta	String	Path HTTP della richiesta SOAP ricevuta dal SIL
httpStatusRisposta	Integer	Codice di stato HTTP della risposta dal backend (null se non disponibile)
esito	String	Esito della transazione: "OK" o "ERRORE"
messaggioErrore	String	Messaggio di errore (solo se esito = ERRORE, senza dati sensibili)
durataMs	Long	Durata della transazione in millisecondi
timestampRichiesta	LocalDateTime	Timestamp della richiesta SOAP ricevuta dal middleware

Nota: Il logging è sincrono (post-request) ma non bloccante: se l'inserimento in DB fallisce, viene registrato un warning nel log applicativo senza interrompere la risposta al SIL.

Repository Jdbi (repository/)

EnteRepository.java (DAO Jdbi)

Tipo: interface con annotazioni Jdbi (@SqlQuery)

Scopo: Accesso in lettura alla tabella `mygov_ente`. Registrato come bean Spring tramite `JdbiConfiguration`.

Query principale: LEFT JOIN con mygov_ente_config_pu per popolare la cache con EnteCompleto in un'unica operazione. Include il campo codice_fiscale_ente in tutte le SELECT.

Metodi (4):

Metodo	Return	Descrizione
findByCodIpaEnte(String)	Optional<Ente>	Cerca per codice IPA
findByCodiceFiscale(String)	Optional<Ente>	Cerca per codice fiscale (WHERE codice_fiscale_ente = :codiceFiscaleEnte)
findAll()	List<Ente>	Tutti gli enti ordinati per codice IPA
count()	long	Conteggio totale enti

EnteConfigPuRepository.java (DAO Jdbi)

Tipo: interface con annotazioni Jdbi (@SqlQuery , @SqlUpdate)

Scopo: Accesso ai dati per la tabella mygov_ente_config_pu . Registrato come bean Spring tramite JdbiConfiguration .

Operazioni principali:

Metodo	Tipo SQL	Descrizione
findByCodIpaEnte(codIpaEnte)	@SqlQuery	Recupera la configurazione PU di un ente specifico
findAllActive()	@SqlQuery	Tutte le configurazioni attive — usata per popolare la cache
insert(codIpaEnte, clientId, clientSecret)	@SqlUpdate	Inserisce una nuova configurazione PU per un ente
updateAttivo(codIpaEnte, attivo)	@SqlUpdate	Abilita/disabilita un ente per la PU

TransactionLogRepository.java (DAO Jdbi)

Tipo: interface con annotazioni Jdbi

Scopo: Scrittura log transazionale nella tabella mygov_mw_transaction_log . Registrato come bean Spring tramite JdbiConfiguration .

Metodi (1):

Metodo	Tipo SQL	Descrizione
insert(codIpaEnte, tipoOperazione, modalitaRouting, destinazione, pathRichiesta, httpStatusRisposta, esito, messaggioErrore, durataMs)	@SqlUpdate	Inserisce un record di log — 9 parametri via @Bind

Nota: Le operazioni di lettura (reporting, diagnostica) potranno essere aggiunte in futuro. Attualmente il focus è sull'inserimento sincrono post-request.

Cache in-memory (EnteCacheService.java)

Tipo: @Service

Scopo: Mantiene una copia in-memory di mygov_ente LEFT JOIN mygov_ente_config_pu con un TTL configurabile, evitando query al DB a ogni richiesta SOAP. Implementa una **cache duale** per supportare l'identificazione dell'ente sia per codIpaEnte che per codiceFiscaleEnte (usato come identificativoDominio nelle richieste SOAP FESP).

Struttura interna:

Componente	Tipo	Scopo
------------	------	-------

cacheByCodIpa	ConcurrentHashMap<String, EnteCompleto>	Mappa thread-safe codIpaEnte → EnteCompleto (lookup primario)
cacheByCodiceFiscale	ConcurrentHashMap<String, EnteCompleto>	Mappa thread-safe codiceFiscaleEnte → EnteCompleto (lookup secondario — solo enti con codice fiscale non nullo)
ultimoCaricamento	volatile Instant	Timestamp dell'ultimo refresh (inizializzato a Instant.EPOCH)
refreshLock	ReentrantLock	Garantisce che un solo thread alla volta esegua il refresh

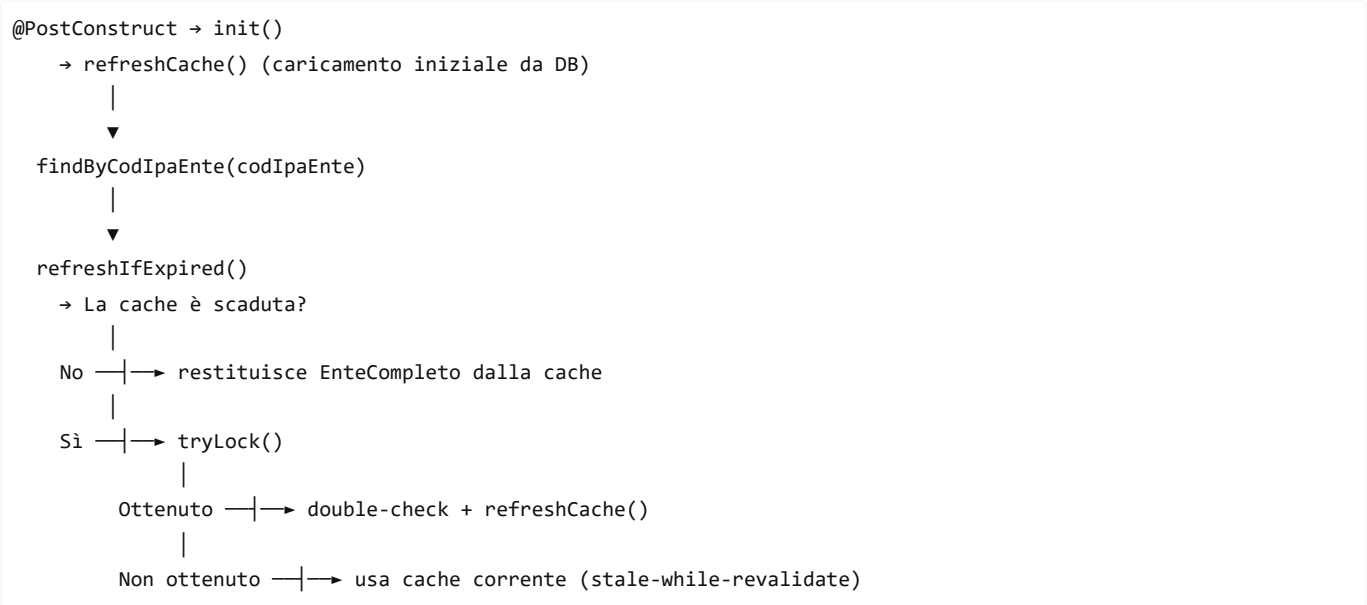
Chiavi cache:

- Primaria: `codIpaEnte` (es. "R_LOMBARDIA")
- Secondaria: `codiceFiscaleEnte` (es. "80007580279") — usata quando la richiesta SOAP contiene `<identificativoDominio>` al posto di `<codIpaEnte>`

TTL configurabile:

```
middleware.cache.ente-config.ttl-seconds=300 # default: 5 minuti – riferisce mygov_ente_config_pu
```

Ciclo di vita:



Metodi pubblici:

Metodo	Descrizione
<code>findByCodIpaEnte(codIpaEnte)</code>	Recupera EnteCompleto dalla cache primaria (con refresh se TTL scaduto)
<code>findByCodiceFiscale(codiceFiscale)</code>	Recupera EnteCompleto dalla cache secondaria per codice fiscale (con refresh se TTL scaduto)
<code>size()</code>	Numero di entry in cache — usato dal Gauge Micrometer e dall'health check
<code>countEntiPiattaformaUnitaria()</code>	Numero di enti con <code>isPiattaformaUnitaria() == true</code> — usato dall'health check
<code>forceRefresh()</code>	Forza il ricaricamento della cache indipendentemente dal TTL

Pattern stale-while-revalidate: Se il refresh fallisce (es. errore DB), la cache corrente viene mantenuta e l'errore viene registrato nel log applicativo. Questo garantisce che il middleware continui a funzionare anche in caso di temporanea indisponibilità del database.

9. Modulo Routing

Questo modulo implementa la logica di decisione del routing — il "cervello" del gateway. Determina dove instradare ogni richiesta SOAP in base a due dimensioni: il path HTTP (per identificare il backend di destinazione) e la configurazione dell'ente nel database (per determinare la modalità di instradamento).

`RoutingDecision.java` (risultato immutabile)

Tipo: Classe immutabile

Scopo: Contiene tutte le informazioni necessarie all'endpoint SOAP per instradare una richiesta.

Campi (4):

Campo	Tipo	Descrizione
destinazione	BackendDestinatario	Backend di destinazione (MYPAY O MYPIVOT), determinato dal path HTTP
modalita	ModalitaRouting	Modalità di instradamento (PIATTAFORMA_UNITARIA O LEGACY), derivata da <code>EnteCompleto.isPiattaformaUnitaria()</code>
urlBackend	String	URL base del backend, determinato da <code>BackendRoutingConfig</code>
ente	EnteCompleto	Oggetto aggregato dell'ente — contiene credenziali OAuth2 usate da <code>PiattaformaUnitariaClient</code>

Metodi di convenienza:

Metodo	Descrizione
<code>isPiattaformaUnitaria()</code>	<code>true</code> se <code>modalita == PIATTAFORMA_UNITARIA</code>
<code>isLegacy()</code>	<code>true</code> se <code>modalita == LEGACY</code>
<code>getDestinazione()</code>	Restituisce il backend di destinazione
<code>getModalita()</code>	Restituisce la modalità di instradamento
<code>getUrlBackend()</code>	Restituisce l'URL base del backend
<code>getEnte()</code>	Restituisce l' <code>EnteCompleto</code> con le credenziali OAuth2 per-ente

`RoutingDecisionService.java` (servizio di decisione)

Tipo: `@Service`

Scopo: Servizio centrale di decisione del routing. Data una richiesta SOAP (identificata da `codIpaEnte` e `pathRichiesta`), produce una `RoutingDecision`.

Dipendenze (3, iniettate via costruttore):

Dipendenza	Tipo	Scopo
<code>pathRegistryConfig</code>	<code>PathRegistryConfig</code>	Risolve il path HTTP → backend di destinazione
<code>enteCacheService</code>	<code>EnteCacheService</code>	Recupera l' <code>EnteCompleto</code> dalla cache/DB
<code>backendRoutingConfig</code>	<code>BackendRoutingConfig</code>	Fornisce l'URL base del backend di destinazione

Algoritmo di decisione a 2 passi (semplificato rispetto ai 3 passi precedenti):

```

decide(codIpaEnte, pathRichiesta)
    |
    ▼
    Passo 1: Routing per path → destinazione backend
    pathRegistryConfig.resolveBackend(pathRichiesta)
    |
    Non trovato → PathNonRiconosciutoException
                  → SOAP Fault PATH_NON_RICONOSCIUTO (Client Fault)
    |
    Trovato → BackendDestinatario (MYPAY o MYPIVOT)
    |
    ▼
    Passo 2: Recupera EnteCompleto dalla cache
    enteCacheService.findByCodIpaEnte(codIpaEnte)
    |
    Non trovato → EnteNonCensitoException
                  → SOAP Fault ENTE_NON_AUTORIZZATO (Client Fault)
    |
    Trovato → EnteCompleto (con credenziali OAuth2 se PU)
    |
    ▼
    Componi la decisione:
    modalita = ente.isPiattaformaUnitaria() ? PIATTAFORMA_UNITARIA : LEGACY
    urlBackend = backendRoutingConfig.getBaseUrlFor(destinazione)
    |
    ▼
    return RoutingDecision(destinazione, modalita, urlBackend, ente)

```

Importante: Il servizio **non esegue alcuna comunicazione HTTP** — si limita a prendere la decisione. L'effettivo inoltra è responsabilità degli endpoint SOAP (sottoclassi di `AbstractSoapProxyEndpoint`) che utilizzano `PiattaformaUnitariaClient` o `ProxyForwardingClient` in base alla decisione.

10. Modulo Endpoint SOAP

Il middleware espone **40 operazioni SOAP** distribuite su **10 endpoint** concreti, tutti basati su una classe astratta comune (`AbstractSoapProxyEndpoint`). Ogni endpoint è un **proxy trasparente**: riceve la richiesta SOAP dal SIL, identifica l'ente, determina il routing e inoltra l'intero Envelope al backend appropriato.

Panoramica degli endpoint

#	Classe	Namespace	Ops	Path SIL	
1	PagamentiTelematiciDovutiPagatiEndpoint	http://www.regione.veneto.it/pagamenti/ente/	16	/ws/pa	
2	PagamentiTelematiciCCPPaEndpoint	http://www.regione.veneto.it/pagamenti/pa/	4	/ws/pa	
3	PagamentiTelematiciEsitoEndpoint	http://www.regione.veneto.it/pagamenti/pa/	1	/ws/pa	
4	PagamentiTelematiciFlussiSPCEndpoint	http://www.regione.veneto.it/pagamenti/pa/	2	/ws/pa	
5	PagamentiTelematiciRPEndpoint	http://www.regione.veneto.it/pagamenti/nodoregionalefesp/	8	/ws/fesp	
6	PagamentiTelematiciCCP25Endpoint	http://pagopa-api.pagopa.gov.it/pa/paForNode.xsd	5	/ws/fesp	
7	PagamentiTelematiciCCPEndpoint	http://ws.pagamenti.telematici.gov/	2	/ws/fesp	
8	PagamentiTelematiciRTEndpoint	http://ws.pagamenti.telematici.gov/	1	/ws/fesp	

9	PagamentiTelematiciAvvisiDigitaliEndpoint	http://www.regione.veneto.it/pagamenti/nodoregionalefesp/	1	/ws/fesp
10	ReconciliationEndpoint	http://www.regione.veneto.it/pagamenti/pivot/ente/	10	/ws/pivot

AbstractSoapProxyEndpoint.java (classe base)

Tipo: abstract class

Scopo: Classe base comune a tutti i 10 endpoint. Implementa il pattern "proxy SOAP trasparente" con identificazione ente duale, routing dinamico, logging e metriche. Le sottoclassi concreti aggiungono solo la mappatura namespace/localPart tramite `@PayloadRoot`.

Dipendenze iniettate (6, tutte via costruttore):

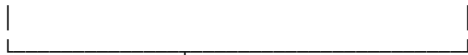
Dipendenza	Tipo	Scopo
piattaformaClient	PiattaformaUnitariaClient	Inoltro verso la PU con OAuth2
proxyForwardingClient	ProxyForwardingClient	Forward trasparente verso i backend legacy
routingDecisionService	RoutingDecisionService	Decisione di routing (path + DB → destinazione)
transactionLoggingService	TransactionLoggingService	Log transazionale su DB
metricsService	MiddlewareMetricsService	Raccolta metriche Micrometer
enteCacheService	EnteCacheService	Cache duale per la risoluzione ente (codIpa + codiceFiscale)

Metodo centrale `processRequest(Element request, MessageContext messageContext, String platformPath)` :

Ogni metodo `@PayloadRoot` delle sottoclassi delega interamente a `processRequest()`, passando il path della PU come costante. Il flusso è:



```
fullSoapEnvelope, requestPath, fullSoapEnvelope)
decision.getEnte())
```



5. Estrae il contenuto del Body dalla risposta SOAP
(extractBodyContent: cerca <Body> per namespace
→ restituisce primo Element figlio)



6. Registra successo:
- transactionLoggingService.logSuccesso(...)
- metricsService.registraSuccesso(...)



7. Restituisce l'Element a Spring WS
(Spring WS lo ri-avvolge in un nuovo SOAP Envelope)

Identificazione ente duale — extractEnteIdentifier(String soapEnvelope) :

La richiesta SOAP può contenere l'identificativo dell'ente in due formati diversi, a seconda del namespace/endpoint:

1. **<codIpaEnte>** — usato dagli endpoint PA e MyPivot. Il valore è direttamente il codice IPA dell'ente.
2. **<identificativoDominio>** — usato dagli endpoint FESP. Il valore è il **codice fiscale** dell'ente, che viene risolto in **codIpaEnte** tramite `enteCacheService.findByCodiceFiscale()` .

```
extractEnteIdentifier(soapEnvelope)
```



Cerca <codIpaEnte> nell'Envelope (getElementsByTagName, namespace-agnostic)



Trovato → restituisce il valore direttamente (è già il codice IPA)



Non trovato → cerca <identificativoDominio> nell'Envelope



Trovato → `enteCacheService.findByCodiceFiscale(identificativoDominio)`



→ risolve il codice fiscale in **codIpaEnte**

→ lancia `IllegalStateException` se non trovato

Non trovato → lancia `IllegalStateException`

("Impossibile identificare l'ente dalla richiesta SOAP")

Gestione errori:

- **Errori pre-routing** (ente non censito, path non riconosciuto): usa `logErrorePreRouting()` con `modalitaRouting="SCONOSCIUTA"` e `destinazione="SCONOSCIUTA"`
- **Errori post-routing** (comunicazione fallita, timeout): usa `logErrore()` con la `RoutingDecision` già calcolata
- In entrambi i casi: registra l'errore nelle metriche con `metricsService.registraErrore()`
- Le eccezioni `RuntimeException` vengono propagate al `SoapFaultExceptionResolver` ; le altre vengono avvolte in `RuntimeException`

Sicurezza XML (prevenzione attacchi XXE): Il parser XML è configurato con tutte le protezioni contro gli attacchi XXE (XML External Entity):

- `disallow-doctype-decl: true` — blocca le dichiarazioni DTD
- `external-general-entities: false` — disabilita le entità esterne
- `external-parameter-entities: false` — disabilita le entità di parametro esterne

- `XIncludeAware: false` — disabilita XInclude
- `expandEntityReferences: false` — non espande i riferimenti a entità
- `TransformerFactory` : attributi `ACCESS_EXTERNAL_DTD` e `ACCESS_EXTERNAL_STYLESHEET` impostati a stringa vuota

Struttura delle sottoclassi concrete

Ogni endpoint concreto segue lo stesso pattern uniforme:

```
@Endpoint
public class NomeEndpoint extends AbstractSoapProxyEndpoint {

    private static final String NAMESPACE_URI = "...";
    private static final String PLATFORM_PATH = Constants.PLATFORM_PATH_PU_MYPAY; // o PLATFORM_PATH_PU_MYPIVOT

    // Costruttore con le stesse 6 dipendenze

    @PayloadRoot(namespace = NAMESPACE_URI, localPart = "nomeOperazione")
    @ResponsePayload
    public Element handleNomeOperazione(@RequestPayload Element request,
                                         MessageContext messageContext) {
        return processRequest(request, messageContext, PLATFORM_PATH);
    }

    @Override
    protected String getDefaultPath() { return "/ws/pa"; } // o /ws/fesp, /ws/pivot

    /**
     * Restituisce il namespace da usare nel dettaglio del SOAP Fault per questo endpoint.
     * Gli endpoint MyPay restituiscono NS_FAULT_MYPAY; l'endpoint MyPivot restituisce NS_FAULT_MYPIVOT.
     */
    @Override
    protected String getFaultDetailNamespace() { return Constants.NS_FAULT_MYPAY; }
}
```

Il metodo `getFaultDetailNamespace()` è **obbligatorio** su tutte le sottoclassi. Permette al `SoapFaultExceptionResolver` di costruire il `<detail>` del SOAP Fault con il namespace corretto per ciascun endpoint, eliminando il namespace hardcoded `MyPivot` che veniva erroneamente usato per tutti gli endpoint (fix critico del 31 Mar 2026).

Esempio: `ReconciliationEndpoint.java` (`MyPivot`)

L'endpoint `MyPivot` espone 10 operazioni, tutte con lo stesso namespace e `PLATFORM_PATH` (area riconciliazione), di cui una con post-processing per il proxy upload:

- Namespace: `http://www.regione.veneto.it/pagamenti/pivot/ente/`
- Path SIL: `/ws/pivot/PagamentiTelematiciPagatiRiconciliati`
- Path PU: `/pu/sil/soap/reconciliation/PagamentiTelematiciPagatiRiconciliati`
- Operazioni: `pivotSILAutorizzaImportFlusso` (con post-processing proxy upload), `pivotSILAutorizzaImportFlussoRendicontazione`, `pivotSILAutorizzaImportFlussoRT`, `pivotSILAutorizzaImportFlussoTesoreria`, `pivotSILChiediAccertamento`, `pivotSILChiediPagatiRiconciliati`, `pivotSILChiediStatoExportFlussoRiconciliazione`, `pivotSILChiediStatoImportFlusso`, `pivotSILChiediStatoImportFlussoTesoreria`, `pivotSILPrenotaExportFlussoRiconciliazione`

Dettaglio operazioni per endpoint

PagamentiTelematiciDovutiPagatiEndpoint (16 operazioni): `paaSILImportaDovuto`, `paaSILAutorizzaImportFlusso`, `paaSILChiediEsitoCarrelloDovuti`, `paaSILChiediPagati`, `paaSILChiediPagatiConRicevuta`, `paaSILChiediPosizioniAperte`, `paaSILChiediStatoExportFlusso`, `paaSILChiediStatoImportFlusso`, `paaSILChiediStoricoPagamenti`, `paaSILInviaDovuti`,

paaSILInviaCarrelloDovuti , paaSILPrenotaExportFlusso , paaSILPrenotaExportFlussoIncrementaleConRicevuta ,
paaSILRegistraPagamento , paaSILVerificaAvviso , paaSILRecuperaAvviso

PagamentiTelematiciCCPPaEndpoint (4 operazioni): paaSILAttivaRP , paaSILVerificaRP , paVerifyPaymentNotice ,
paGetPayment

PagamentiTelematiciEsitoEndpoint (1 operazione): paaSILInviaEsito

PagamentiTelematiciFlussiSPCEndpoint (2 operazioni): paaSILChiediFlussoSPC , paaSILChiediElencoFlussiSPC

PagamentiTelematiciRPEndpoint (8 operazioni): chiediFlussoSPC , chiediFlussoSPCPage , chiediListaFlussiSPC ,
nodoSILChiediCopiaEsito , nodoSILInviaRP , nodoSILChiediIUV , nodoSILInviaCarrelloRP , nodoSILRichiediRT

PagamentiTelematiciCCP25Endpoint (5 operazioni): paVerifyPaymentNoticeReq , paGetPaymentReq , paSendRTReq ,
paSendRTV2Request , paGetPaymentV2Request

PagamentiTelematiciCCPEndpoint (2 operazioni): paaVerificaRPT , paaAttivaRPT

PagamentiTelematiciRTEndpoint (1 operazione): paaInviaRT

PagamentiTelematiciAvvisiDigitaliEndpoint (1 operazione): nodoSILInviaAvvisoDigitale

ReconciliationEndpoint (10 operazioni): pivotSILAutorizzaImportFlusso , pivotSILAutorizzaImportFlussoRendicontazione ,
pivotSILAutorizzaImportFlussoRT , pivotSILAutorizzaImportFlussoTesoreria , pivotSILChiediAccertamento ,
pivotSILChiediPagatiRiconciliati , pivotSILChiediStatoExportFlussoRiconciliazione ,
pivotSILChiediStatoImportFlusso , pivotSILChiediStatoImportFlussoTesoreria ,
pivotSILPrenotaExportFlussoRiconciliazione

NOTA: Questo approccio "proxy trasparente" è necessario perché la PU richiede l'Header SOAP con `codIpaEnte` (o
`identificativoDominio`) per identificare l'ente. Se si inoltrasse solo il Body (come farebbe un `@PayloadRoot` standard), la PU non
saprebbe quale ente sta effettuando la richiesta.

Approccio contract-last: Gli endpoint sono implementati senza un WSDL predefinito. Il contratto è definito dal codice Java (namespace + localPart). In fasi future si potrà migrare a contract-first con generazione da WSDL/XSD.

11. Proxy Upload Flusso Import

Questa funzionalità risolve un problema specifico dei flussi di import: la Piattaforma Unitaria (e il backend legacy) restituiscono, in risposta alle operazioni `paaSILAutorizzaImportFlusso` (MyPay) e `pivotSILAutorizzaImportFlusso` (MyPivot), un URL temporaneo (`uploadUrl`) al quale il SIL deve poi caricare il file. Il SIL però non è in grado di gestire l'autenticazione OAuth2 (richiesta dalla PU) né conosce la differenza tra modalità PU e LEGACY. Il middleware si interpone come proxy trasparente anche per questo secondo passo dell'upload.

Flusso complessivo

Passo 1 – Autorizzazione (SOAP)

```
SIL → POST /ws/pa/PagamentiTelematiciDovutiPagati (MyPay)
      oppure POST /ws/pivot/PagamentiTelematiciPagatiRiconciliati (MyPivot)
Body: <paaSILAutorizzaImportFlusso>...</paaSILAutorizzaImportFlusso>
      oppure <pivotSILAutorizzaImportFlusso>...</pivotSILAutorizzaImportFlusso>
      |
      ▼
PagamentiTelematiciDovutiPagatiEndpoint (MyPay)
oppure ReconciliationEndpoint (MyPivot)
(post-processing attivo solo per le operazioni di autorizzazione import flusso)
      |
      ▼
```

Middleware → Backend (PU o LEGACY)

|

← risposta backend:

uploadUrl (es. https://api.uat.p4pa.pagopa.it/pu/fileshare/...)
authorizationToken (per LEGACY: token univoco; per PU: valore costante/non-univoco)
requestToken (sempre univoco, UUID generato dal backend)
importPath

|

▼ post-processing:

a. Determina la chiave di cache in base alla modalità di routing:

→ se modalità PIATTAFORMA_UNITARIA:

chiaveCache = requestToken (univoco)
sostituisce <authorizationToken> nella risposta XML con requestToken
(il SIL riceverà requestToken al posto dell'authorizationToken non univoco)

→ se modalità LEGACY:

chiaveCache = authorizationToken (univoco reale del backend)

b. Salva in UploadProxyCacheService:

chiaveCache → UploadProxyEntry {
uploadUrl originale, modalitaRouting, codIpaEnte,
authorizationTokenSil, requestToken, endpointOrigine (MYPAY o MYPIVOT)
}

c. Sostituisce uploadUrl nella risposta XML con:

\${middleware.upload.proxy.base-url}/api/upload/flusso

|

▼

SIL riceve la risposta modificata:

uploadUrl = http://localhost:8086/api/upload/flusso
authorizationToken = <requestToken> (se PU) o <token-originale> (se LEGACY)

Passo 2 – Upload file (REST multipart)

SIL → POST /api/upload/flusso

Content-Type: multipart/form-data

Parametri: authorizationToken=<token>, file=<contenuto binario>

|

▼

UploadFlussoController

→ UploadProxyCacheService.recuperaERimuovi(authorizationToken)

→ recupera UploadProxyEntry (one-shot: rimossa dalla cache dopo il prelievo)

→ il parametro authorizationToken viene usato come chiave di cache:

per PU è il requestToken, per LEGACY è il token reale

→ se non trovata → HTTP 400 (token non valido o scaduto)

|

▼

Se la entry ha endpointOrigine == MYPAY e modalità PU:

→ validazione versione file tramite UpdateFilePuService

→ se non supportata: errore applicativo PAA_IMPORT_FILE_VERSIONE_ERR

Se la entry ha endpointOrigine == MYPIVOT e modalità PU:

→ validazione versione file SALTATA (non applicabile per flussi MyPivot)

Inoltra a UploadForwardingClient:

→ se modalità LEGACY:

POST multipart verso uploadUrl originale

Parametri: authorizationToken (dal SIL) + file

→ se modalità PIATTAFORMA_UNITARIA:

POST multipart verso uploadUrl originale

Parametri: authorizationToken (dal SIL) + file

Header: Authorization: Bearer <token-0Auth2-per-ente>

|



Risposta del backend → restituita as-is al SIL (HTTP status + body)

Componenti implementati

`UploadProxyEntry.java` (DTO immutabile)

Tipo: Classe immutabile (POJO con costruttore)

Pacchetto: `api/upload/`

Scopo: Rappresenta la voce della cache di proxy upload. Viene creata durante il post-processing di `paaSILAutorizzaImportFlusso` (MyPay) e `pivotSILAutorizzaImportFlusso` (MyPivot), e consumata (one-shot) da `UploadFlussoController`.

Campi:

Campo	Tipo	Descrizione
<code>uploadUrlOriginale</code>	<code>String</code>	URL originale di upload restituito dal backend (PU o legacy)
<code>authorizationToken</code>	<code>String</code>	Token inviato al SIL nella risposta SOAP e usato per la lookup in cache — per PU contiene <code>requestToken</code> , per LEGACY contiene il token reale del backend
<code>requestToken</code>	<code>String</code>	Token univoco della richiesta (UUID generato dal backend) — usato come chiave di cache per le richieste instradate verso PU
<code>importPath</code>	<code>String</code>	Percorso relativo per l'import del flusso
<code>modalitaRouting</code>	<code>ModalitaRouting</code>	Modalità di routing dell'ente (PU o LEGACY)
<code>codIpaEnte</code>	<code>String</code>	Codice IPA dell'ente — per logging e metriche
<code>endpointOrigine</code>	<code>BackendDestinatario</code>	Endpoint di origine (MYPAY o MYPIVOT) — usato da <code>UploadFlussoController</code> per decidere se applicare la verifica versione file (solo MyPay)
<code>timestampCreazione</code>	<code>Instant</code>	Timestamp di creazione — usato per la pulizia TTL

`UploadProxyCacheService.java` (cache TTL in-memory)

Tipo: `@Service`

Pacchetto: `api/upload/`

Scopo: Mantiene una mappa in-memory `chiaveCache` → `UploadProxyEntry` con TTL configurabile e semantica **one-shot** (ogni entry viene rimossa al primo utilizzo). Una task schedulata pulisce periodicamente le entry scadute.

La chiave della cache è **condizionale** alla modalità di routing:

- **Routing PU:** chiave = `requestToken` (univoco) — perché la PU non restituisce un `authorizationToken` univoco
- **Routing LEGACY:** chiave = `authorizationToken` (univoco reale del backend)

Struttura interna:

Componente	Tipo	Scopo
<code>cache</code>	<code>ConcurrentHashMap<String, UploadProxyEntry></code>	Mappa thread-safe token → entry

Comportamento one-shot: Il metodo `getAndRemove(authorizationToken)` restituisce l'entry e la rimuove contestualmente dalla cache. Questo garantisce che ogni token di upload possa essere usato una sola volta, in analogia con la semantica del token monouso del backend.

Pulizia periodica con `@Scheduled` : Il metodo di pulizia viene invocato automaticamente ogni `middleware.upload.proxy.cleanup-interval-ms` millisecondi (default: 300000 ms = 5 minuti). Rimuove tutte le entry il cui `createdAt` supera il TTL configurato. Questo meccanismo richiede `@EnableScheduling` su `Application.java` .

Properties configurabili:

```
middleware.upload.proxy.cache-ttl-seconds=3600
middleware.upload.proxy.cleanup-interval-ms=300000
```

Metodi pubblici:

Metodo	Descrizione
<code>put(authorizationToken, entry)</code>	Inserisce una nuova entry nella cache
<code>getAndRemove(authorizationToken)</code>	Recupera l'entry e la rimuove (one-shot) — restituisce <code>Optional.empty()</code> se assente o scaduta
<code>size()</code>	Numero di entry attualmente in cache

`UploadFlussoController.java` (endpoint REST)

Tipo: `@RestController`

Pacchetto: `api/upload/`

Scopo: Espone l'endpoint REST `POST /api/upload/flusso` al quale il SIL invia il file dopo aver ricevuto la risposta modificata di `paaSILAutorizzaImportFlusso` .

Dipendenza aggiuntiva nel `pom.xml` : `spring-boot-starter-web` (aggiunto per abilitare `@RestController` in un'applicazione che in precedenza dipendeva solo da `spring-boot-starter-web-services`).

Endpoint:

`POST /api/upload/flusso`

`Content-Type: multipart/form-data`

Parametri:

`authorizationToken` (String, obbligatorio) – token monouso ricevuto dal SIL
`file` (MultipartFile, obbligatorio) – file da caricare

Flusso interno:

- Estrae il file multipart dalla richiesta
→ se assente o vuoto: restituisce errore applicativo compatibile con il backend
- Recupera l'entry dalla cache: `UploadProxyCacheService.recuperaERimuovi(authorizationToken)`
→ se assente: restituisce errore applicativo compatibile con il backend
- Se l'entry è in modalità PU e ha `endpointOrigine == MYPAY`:
 - invoca `UpdateFilePuService.verificaVersionePerPu(file)`
 - se la versione non è supportata: restituisce errore applicativo `PAA_IMPORT_FILE_VERSIONE_ERR(...)`
 - se la versione è supportata: inoltra verso PU tramite `UploadForwardingClient.inoltraAllaPU(...)`
- Se l'entry è in modalità PU e ha `endpointOrigine == MYPIVOT`:
 - verifica versione file SALTATA (non applicabile per flussi MyPivot)
 - inoltra direttamente verso PU tramite `UploadForwardingClient.inoltraAllaPU(...)`
- Se l'entry è in modalità LEGACY: inoltra direttamente tramite `UploadForwardingClient.inoltraAlLegacy(...)`
- Restituisce al SIL la risposta del backend o l'errore applicativo nel formato legacy compatibile

Nota di compatibilità: Anche nei casi di errore applicativo il controller mantiene il comportamento compatibile con il backend legacy, restituendo HTTP 200 con body JSON nel formato `[{codice, descrizione}]` .

Configurazione multipart:

```
spring.servlet.multipart.max-file-size=100MB
spring.servlet.multipart.max-request-size=100MB
```

UpdateFilePuService.java (validazione versione file upload PU)

Tipo: @Service

Pacchetto: api/upload/

Scopo: Valida la versione del tracciato dei file caricati verso Piattaforma Unitaria prima dell'inoltro al backend.

Logica:

- Estrae la versione dal nome file originale nel formato finale `x_y` (es. `1_4`)
- Consente l'inoltro diretto verso PU per le versioni `1_0` , `1_1` , `1_2` , `1_3`
- Riconosce le versioni `1_4` e `1_5` ma le blocca temporaneamente, perché la trasformazione al tracciato `2.0` non è ancora implementata
- Per versione non rilevata o non supportata restituisce un errore applicativo compatibile con il backend legacy

Esito della validazione:

Versione file	Esito
1_0	inoltro consentito
1_1	inoltro consentito
1_2	inoltro consentito
1_3	inoltro consentito
1_4	rifiutata temporaneamente
1_5	rifiutata temporaneamente
altra / assente	rifiutata

Errore restituito:

- Codice: `PAA_IMPORT_FILE_VERSIONE_ERR(<versione>)`
- Descrizione: versione tracciato non supportata e rinvio al manuale "Integrazione Ente"

Dipendenze principali:

- `SupportedFileVersion` — enum che mantiene il mapping tra formato canonico (`1.4`) e formato presente nel nome file (`1_4`)
- `MultipartFile` — sorgente del nome file originale da validare

UploadForwardingClient.java (client HTTP upload)

Tipo: @Service

Pacchetto: api/client/

Scopo: Client HTTP responsabile dell'inoltro del file al backend appropriato (PU o LEGACY), costruendo la richiesta multipart corretta in base alla modalità di routing.

Differenza tra modalità:

Modalità	Header aggiunto	Autenticazione
LEGACY	nessuno	Solo <code>authorizationToken</code> nel multipart

PIATTAFORMA_UNITARIA	Authorization: Bearer <token-OAuth2>	authorizationToken nel multipart + Bearer OAuth2
----------------------	--------------------------------------	--

Per la modalità PU, il Bearer token viene ottenuto chiamando `OAuthTokenService.getAccessToken(ente.getCodIpaEnte(), ente.getClientId(), ente.getClientSecret())`, esattamente come avviene in `PiattaformaUnitariaClient`.

Configurazione HTTP:

```
middleware.upload.proxy.connect-timeout-ms=10000
middleware.upload.proxy.read-timeout-ms=120000
```

Il timeout di lettura è volutamente più alto (120 secondi) rispetto ai client SOAP (30 secondi) per accomodare upload di file di grandi dimensioni.

Post-processing negli endpoint SOAP

File modificati:

- `soap/endpoint/mypay/PagamentiTelematiciDovutiPagatiEndpoint.java` — operazione `paaSILAutorizzaImportFlusso`
- `soap/endpoint/mypivot/ReconciliationEndpoint.java` — operazione `pivotSILAutorizzaImportFlusso`

Il post-processing è attivato **esclusivamente** per le operazioni di autorizzazione import flusso. Tutte le altre operazioni degli endpoint rimangono proxy trasparenti senza modifiche alla risposta.

Logica di post-processing (identica per entrambi gli endpoint, cambia solo il valore di `endpointOrigine`):

```
risposta XML ricevuta dal backend
|
▼
Contiene <uploadUrl> e <authorizationToken>?
|
No —→ restituisce la risposta invariata al SIL (proxy trasparente)
|
Sì —→
|
▼
Estrae uploadUrl, authorizationToken, requestToken, importPath dalla risposta XML
|
▼
Recupera codIpaEnte e modalitaRouting dall'ente
|
▼
Se modalitaRouting == PIATTAFORMA_UNITARIA:
  → chiaveCache = requestToken (univoco)
  → authorizationTokenSil = requestToken
  → sostituisce <authorizationToken> nella risposta XML con requestToken
  → (motivo: la PU restituisce un authorizationToken non univoco,
     requestToken è l'identificatore univoco della sessione)
Se modalitaRouting == LEGACY:
  → chiaveCache = authorizationToken (univoco reale del backend)
  → authorizationTokenSil = authorizationToken
  → nessuna modifica al campo <authorizationToken> nella risposta
|
▼
Crea UploadProxyEntry { uploadUrl, authorizationTokenSil, requestToken,
                        importPath, modalitaRouting, codIpaEnte, endpointOrigine }
|
▼
UploadProxyCacheService.salva(chiaveCache, entry)
```



Sostituisce uploadUrl nella risposta XML con:
`${middleware.upload.proxy.base-url}/api/upload/flusso`



Restituisce la risposta modificata al SIL

Il valore di `endpointOrigine` differisce tra i due endpoint:

- `PagamentiTelematiciDovutiPagatiEndpoint` → `BackendDestinatario.MYPAY`
- `ReconciliationEndpoint` → `BackendDestinatario.MYPIVOT`

Questo permette a `UploadFlussoController` di saltare la verifica della versione del file per le richieste originate da MyPivot.

Properties introdotte

Tutte le properties sono definite nel blocco `middleware.upload.proxy.*` e in `spring.servlet.multipart.*`.

application.properties (configurazione base — valori predefiniti):

```
# URL base del middleware esposto ai SIL per l'upload
middleware.upload.proxy.base-url=${MIDDLEWARE_UPLOAD_BASE_URL:http://localhost:8086}

# TTL delle entry in cache (secondi) – default 1 ora
middleware.upload.proxy.cache-ttl-seconds=3600

# Timeout connessione verso il backend di upload (ms)
middleware.upload.proxy.connect-timeout-ms=10000

# Timeout lettura risposta dall'upload (ms) – alto per file grandi
middleware.upload.proxy.read-timeout-ms=120000

# Intervallo di pulizia delle entry scadute (ms) – default 5 minuti
middleware.upload.proxy.cleanup-interval-ms=300000

# Limite dimensione file e richiesta multipart
spring.servlet.multipart.max-file-size=100MB
spring.servlet.multipart.max-request-size=100MB
```

application-dev.properties (override per sviluppo):

```
# TTL ridotto in dev per facilitare i test (10 minuti invece di 1 ora)
middleware.upload.proxy.cache-ttl-seconds=600
```

Modifiche ad `Application.java`

```
@SpringBootApplication
@EnableScheduling // ← aggiunto per abilitare @Scheduled in UploadProxyCacheService
public class Application { ... }
```

`@EnableScheduling` è necessario per attivare il meccanismo di pulizia periodica delle entry scadute nella `UploadProxyCacheService`.

12. Gestione degli Errori

SoapFaultExceptionResolver.java

Tipo: `@Component` + `EndpointExceptionHandler` + `Ordered`

Scopo: Intercetta tutte le eccezioni non gestite negli endpoint SOAP e le converte in **SOAP Fault** strutturate, garantendo che i SIL ricevano sempre una risposta SOAP valida anche in caso di errore.

Priorità: Implementa `Ordered` con `getOrder() = Ordered.HIGHEST_PRECEDENCE`, garantendo precedenza assoluta sui resolver di default di Spring WS.

Mapping delle eccezioni:

Eccezione	SOAP Fault	Codice errore (FaultCode)
EnteNonCensitoException	Client/Sender Fault	ENTE_NON_AUTORIZZATO
EnteNonIdentificabileException	Client/Sender Fault	ENTE_NON_IDENTIFICABILE
CredenzialeSilNonValidaException	Client/Sender Fault	CREDENZIALE_NON_VALIDA
PathNonRiconosciutoException	Client/Sender Fault	PATH_NON_RICONOSCIUTO
PiattaformaAuthenticationException	Server/Receiver Fault	AUTH_ERROR
PiattaformaCommunicationException	Server/Receiver Fault	COMM_ERROR
Qualsiasi altra Exception	Server/Receiver Fault	INTERNAL_ERROR

Logica di classificazione: Le eccezioni di routing e identificazione ente (`EnteNonCensitoException`, `EnteNonIdentificabileException`, `PathNonRiconosciutoException`) e di credenziali (`CredenzialeSilNonValidaException`) generano Fault **Client** perché rappresentano errori del chiamante. Le eccezioni di comunicazione e autenticazione verso la PU generano Fault **Server** perché rappresentano errori interni al middleware o ai backend.

Namespace dinamico: Il namespace del `<detail>` del Fault viene risolto chiamando `getFaultDetailNamespace()` sull'endpoint che ha gestito la richiesta (ottenuto dal `MethodEndpoint.getBean()`). In questo modo ogni endpoint usa il proprio namespace WSDL:

- Endpoint MyPay → `Constants.NS_FAULT_MYPAY`
- Endpoint MyPivot → `Constants.NS_FAULT_MYPIVOT`

Struttura del SOAP Fault restituito al SIL (esempio per un endpoint MyPay):

```
<soapenv:Fault>
  <faultcode>env:Server</faultcode>
  <faultstring xml:lang="it">Errore di comunicazione con la Piattaforma Unitaria: ...</faultstring>
  <detail>
    <fault:errorCode xmlns:fault="http://www.regione.veneto.it/pagamenti/pa/fault">
      COMM_ERROR
    </fault:errorCode>
  </detail>
</soapenv:Fault>
```

FaultCode.java (31 Mar 2026)

Tipo: `enum`

Scopo: Centralizza tutti i codici di errore SOAP Fault in un'unica enumerazione, eliminando le stringhe hardcoded sparse nel `SoapFaultExceptionResolver`.

Valori:

Valore enum	Codice stringa	Tipo fault
ENTE_NON_AUTORIZZATO	"ENTE_NON_AUTORIZZATO"	Client
ENTE_NON_IDENTIFICABILE	"ENTE_NON_IDENTIFICABILE"	Client
CREDENZIALE_NON_VALIDA	"CREDENZIALE_NON_VALIDA"	Client
PATH_NON_RICONOSCIUTO	"PATH_NON_RICONOSCIUTO"	Client
AUTH_ERROR	"AUTH_ERROR"	Server
COMM_ERROR	"COMM_ERROR"	Server
INTERNAL_ERROR	"INTERNAL_ERROR"	Server

PiattaformaAuthenticationException.java

Tipo: RuntimeException

Quando viene lanciata:

- Credenziali OAuth2 non valide
- Endpoint di autenticazione non raggiungibile
- Risposta senza access_token
- Autenticazione fallita anche dopo il refresh del token

PiattaformaCommunicationException.java

Tipo: RuntimeException con campo aggiuntivo HttpStatus

Quando viene lanciata:

- Timeout nella comunicazione HTTP
- Errore di rete (connessione rifiutata, DNS non risolvibile)
- Risposta HTTP 4xx o 5xx (diversa da 401)
- Circuit breaker aperto (sempre con HttpStatus = 503)
- SOAP Fault ricevuto dal backend (intercettato e convertito in PiattaformaCommunicationException)

Il campo HttpStatus permette al SoapFaultExceptionResolver di includere il codice HTTP nel messaggio di errore restituito al SIL. Il circuit breaker imposta **sempre HTTP 503** (Service Unavailable), indipendentemente dal codice HTTP dell'errore originale che ha causato l'apertura del breaker.

Intercettazione SOAP Fault dal backend (31 Mar 2026): AbstractSoapProxyEndpoint.processRequest() verifica la risposta del backend prima di restituirla al SIL. Se la risposta contiene un <Fault> SOAP, viene estratto il testo del <faultstring> e lanciata una PiattaformaCommunicationException , evitando che namespace e struttura interna del backend vengano esposti ai SIL.

EnteNonCensitoException.java (Fase 7)

Tipo: RuntimeException con campo aggiuntivo codIpaEnte

Quando viene lanciata:

- L'ente non è presente nella tabella mygov_ente con una configurazione attiva in mygov_ente_config_pu
- L' EnteCompleto non è trovato dalla EnteCacheService

SOAP Fault generato: Client/Sender Fault con codice ENTE_NON_AUTORIZZATO

EnteNonIdentificabileException.java (31 Mar 2026)

Tipo: RuntimeException

Quando viene lanciata:

- L'header SOAP della richiesta non contiene né `codIpaEnte` né `identificativoDominio` utilizzabili per identificare l'ente
- L' `AbstractSoapProxyEndpoint` non riesce a estrarre un identificativo ente valido dalla richiesta (sostituisce il precedente `IllegalStateException` che generava erroneamente un Fault **Server**)

SOAP Fault generato: `Client/Sender Fault` con codice `ENTE_NON_IDENTIFICABILE`

Nota: Prima di questo fix (31 Mar 2026), questa condizione lanciava una `IllegalStateException` che veniva mappata come `INTERNAL_ERROR` (Fault Server), inducendo erroneamente il SIL a ritenere il problema lato server.

`PathNonRiconosciutoException.java` (Fase 7)

Tipo: `RuntimeException` con campo aggiuntivo `requestPath`

Quando viene lanciata:

- Il path HTTP della richiesta non corrisponde a nessun backend configurato nel `PathRegistryConfig`

SOAP Fault generato: `Client/Sender Fault` con codice `PATH_NON_RICONOSCIUTO`

13. Monitoraggio e Health Check

Il middleware espone endpoint di monitoraggio tramite **Spring Boot Actuator**.

URL degli endpoint Actuator

Endpoint	URL	Descrizione
Health	GET <code>/actuator/health</code>	Stato generale del sistema
Info	GET <code>/actuator/info</code>	Informazioni applicazione
Metrics	GET <code>/actuator/metrics</code>	Metriche JVM e applicative
Circuit Breakers	GET <code>/actuator/circuitbreakers</code>	Stato dei circuit breaker
Retries	GET <code>/actuator/retries</code>	Statistiche dei retry

`OAuthTokenHealthIndicator.java`

Tipo: `@Component` + `HealthIndicator`

Scopo: Verifica lo stato dei token OAuth2 in cache per tutti gli enti.

Logica (aggiornata):

- Itera su tutti gli enti presenti nella cache token (`OAuthTokenService.getEntiInCache()`)
- **UP** : almeno un ente con token valido; riporta `tokensValidi` e `tokensInCache`
- **DOWN** : nessun token in cache o token scaduti (il middleware li rinnoverà automaticamente al prossimo utilizzo — questo stato è normale al primo avvio)

`PiattaformaUnitariaHealthIndicator.java`

Tipo: `@Component` + `HealthIndicator`

Scopo: Verifica la raggiungibilità della Piattaforma Unitaria.

Logica:

- Effettua una GET leggera verso la base URL della Piattaforma Unitaria
 - Timeout ridotti: connect 3s, read 5s
 - **UP** : la piattaforma risponde
 - **DOWN** : timeout, errore di rete, o risposta di errore
-

EnteConfigHealthIndicator.java (Fase 9)

Tipo: @Component + HealthIndicator

Scopo: Verifica che nel sistema siano configurati enti attivi per il routing.

Logica (aggiornata — usa EnteCacheService):

- Interroga EnteCacheService.size() per ottenere il numero totale di enti in cache
- Interroga EnteCacheService.countEntiPiattaformaUnitaria() per gli enti con PU abilitata
- UP : se entiTotali > 0 — almeno un ente censito e attivo
- DOWN : se la cache è vuota (nessun ente configurato)
- DOWN : se si verifica un'eccezione durante il controllo (errore DB)
- I dettagli esposti: entiTotali , entiConPiattaformaUnitaria , entiLegacy
- DOWN : se la cache è vuota (nessun ente configurato)
- DOWN : se si verifica un'eccezione durante il controllo (errore DB)
- Il numero di enti configurati viene esposto come dettaglio nell'health check

Esempio di risposta Actuator:

```
{
  "status": "UP",
  "details": {
    "entiTotali": 5,
    "entiConPiattaformaUnitaria": 3,
    "entiLegacy": 2
  }
}
```

MiddlewareMetricsService.java (Fase 9)

Tipo: @Service

Scopo: Espone metriche operative del middleware tramite Micrometer, consultabili da Spring Boot Actuator (/actuator/metrics).

Metriche registrate:

Metrica	Tipo Micrometer	Tag	Descrizione
middleware.richieste.totali	Counter	ente, modalita, destinazione, esito	Conteggio richieste per combinazione (rimosso tag operazione)
middleware.richieste.durata	Timer	modalita, destinazione	Distribuzione durata richieste (ms)
middleware.enti.totali	Gauge	—	Numero totale di enti in cache (collegato a EnteCacheService.size())
middleware.enti.piattaforma.unitaria	Gauge	—	Numero di enti abilitati per la PU (collegato a EnteCacheService.countEntiPiattaformaUnitaria())

Metodi principali:

Metodo	Descrizione
registraSuccesso(codIpaEnte, decision, durataMs)	Incrementa contatore con esito=OK e registra durata nel timer


```
registraErrore(codIpaEnte, decision, durataMs)
```

Incrementa contatore con `esito=ERRORE` e registra durata nel timer

Robustezza: I parametri `null` vengono sostituiti con `"sconosciuto"`. Le eccezioni durante la registrazione delle metriche vengono catturate silenziosamente per non bloccare il flusso principale.

Logging applicativo e tecnico

Accanto agli endpoint Actuator, il progetto dispone di una infrastruttura di logging custom che si affianca al logging standard di Spring Boot e ai meccanismi di monitoraggio messi a disposizione da SpringLine2.

La strategia di logging del middleware è divisa in due livelli complementari:

- **logging framework/runtime:** console, root logger, package logger e file gestiti da `logback-spring.xml`
- **logging semantico applicativo:** marker SLF4J centralizzati in `LogMarker.java`, riusati dai componenti per classificare i messaggi

LogMarker.java

`LogMarker` centralizza i marker SLF4J del progetto in un unico enum, evitando stringhe duplicate o incoerenti nei logger applicativi.

Marker enum	Nome marker	Uso previsto
MONITORING	MON_GEN	Eventi di monitoraggio generale
REST	MON_REST	Chiamate REST e integrazioni HTTP
SOAP_SERVER	MON_WSS	Tracciamento richieste SOAP ricevute dal middleware
SOAP_CLIENT	MON_WSC	Chiamate SOAP in uscita verso sistemi esterni
METHOD	MON_METH	Tracing di esecuzione di metodi applicativi
DB_STATEMENT	MON_DBS	Query SQL e tempi di esecuzione
DB_CONNECTION_POOL	MON_CONN	Eventi legati al pool di connessioni

LogHelper.java

`LogHelper` è una utility tecnica che trasforma un oggetto `java.lang.reflect.Method` in una stringa leggibile per i log.

Perché serve:

- rende i log più comprensibili quando il metodo è ottenuto via reflection
- permette di scegliere un formato breve o dettagliato in base al contesto
- viene usata in particolare nel logging SQL Jdbi per indicare quale metodo applicativo ha originato una query

Formati disponibili:

- `methodToShortString(...)` : nome metodo con `(..)` se esistono parametri
- `methodToString(...)` : nome metodo con tipi dei parametri
- `methodToLongString(...)` : modificatore, tipo di ritorno, nome metodo e parametri
- `methodToFullString(...)` : rappresentazione completa con nomi fully-qualified dei tipi e della classe dichiarativa

Logging SQL con `JdbiSqlLogger`

Il layer Jdbi è predisposto per usare `JdbiSqlLogger` come logger SQL personalizzato.

Informazioni tracciate:

- metodo Java sorgente della query
- tempo di esecuzione in millisecondi
- SQL renderizzato
- binding dei parametri quando presenti

- stato read-only della transazione corrente
- evidenziazione delle query lente quando supera la soglia configurata

In caso di eccezione SQL, il logger emette anche lo stack trace associato alla query fallita.

File di log generati

Il file `mypay.mypaycore-properties/src/main/resources/logback-spring.xml` configura i seguenti output:

File	Ruolo
<code>backend.log</code>	Log tecnico generale del backend
<code>mon.log</code>	Tracciamento sintetico di monitoraggio
<code>app.log</code>	Flusso applicativo dedicato
<code>audit.log</code>	Eventi di audit
<code>filter.log</code>	Request/response tracing e filtri

Package `util`

Contiene componenti riusabili e trasversali, da usare per evitare duplicazione di costanti e helper sparsi nel codice.

Classe	Scopo
<code>Constants.java</code>	Contenitore centralizzato delle costanti applicative condivise (namespace, header, codici, chiavi, path, valori ricorrenti)
<code>LogHelper.java</code>	Utility reflection-based che converte <code>Method</code> in firme leggibili per i log, con diversi livelli di dettaglio
<code>Utilities.java</code>	Contenitore di metodi helper stateless e riusabili, richiamabili da più componenti applicativi

Convenzioni di utilizzo:

- `Constants` deve contenere solo costanti condivise e semanticamente stabili; evitare di inserirvi valori temporanei o specifici di una singola classe
- `Utilities` deve ospitare solo logica tecnica riusabile e priva di stato; non deve diventare un contenitore di business logic eterogenea
- quando una costante o una utility è usata da un solo componente, è preferibile mantenerla vicino alla classe che la usa

14. Resilienza

Il middleware implementa la resilienza tramite la libreria **Resilience4j** (versione Spring Boot 3), applicata al metodo `PiattaformaUnitariaClient.forwardSoapRequest()`.

Cos'è Resilience4j e come si usa

Resilience4j è una libreria Java leggera e modulare per la resilienza delle applicazioni distribuite. È progettata per funzionare con programmazione funzionale (lambda) e si integra nativamente con Spring Boot 3 tramite il modulo `resilience4j-spring-boot3`.

La libreria fornisce i seguenti pattern di resilienza:

Pattern	Descrizione
Circuit Breaker	Apre il circuito quando ci sono troppi errori consecutivi, impedendo ulteriori chiamate a un servizio non disponibile. Dopo un periodo di attesa, permette alcune chiamate di test ("half-open") per verificare se il servizio è tornato disponibile.

Retry	Riprova automaticamente le operazioni fallite con diverse strategie di backoff (lineare, esponenziale, fisso).
Rate Limiter	Limita il numero di chiamate in un intervallo di tempo configurabile.
Bulkhead	Isola le risorse per evitare che un errore in un componente si propaghi a tutto il sistema.

Dipendenza Maven

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

La dipendenza `spring-boot-starter-aop` è necessaria perché Resilience4j utilizza AspectJ per intercettare le chiamate ai metodi annotati.

Utilizzo nel codice

I pattern di resilienza si applicano tramite **annotazioni** sui metodi. Nel progetto, il `PiattaformaUnitariaClient.forwardSoapRequest()` è decorato con:

```
@CircuitBreaker(name = "piattaformaUnitaria", fallbackMethod = "forwardSoapRequestFallback")
@Retry(name = "piattaformaUnitaria")
public String forwardSoapRequest(String path, String soapXml) { ... }
```

- `@CircuitBreaker(name = "piattaformaUnitaria")` : Applica il circuit breaker denominato "piattaformaUnitaria". Se il circuit breaker è aperto, non chiama il metodo ma invoca direttamente il metodo di fallback.
- `fallbackMethod = "forwardSoapRequestFallback"` : Specifica il metodo da chiamare quando il circuit breaker è aperto o la chiamata fallisce. Il fallback deve avere la stessa firma del metodo originale più un parametro `Throwable` per l'eccezione.
- `@Retry(name = "piattaformaUnitaria")` : Applica la strategia di retry denominata "piattaformaUnitaria".

Configurazione

La configurazione avviene nel file `application.properties` sotto le chiavi `resilience4j.*` :

```
# Circuit Breaker
resilience4j.circuitbreaker.instances.piattaformaUnitaria.sliding-window-type=COUNT_BASED
resilience4j.circuitbreaker.instances.piattaformaUnitaria.sliding-window-size=10
resilience4j.circuitbreaker.instances.piattaformaUnitaria.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.piattaformaUnitaria.wait-duration-in-open-state=30s
resilience4j.circuitbreaker.instances.piattaformaUnitaria.permitted-number-of-calls-in-half-open-state=3
# Retry
resilience4j.retry.instances.piattaformaUnitaria.max-attempts=3
resilience4j.retry.instances.piattaformaUnitaria.wait-duration=1s
resilience4j.retry.instances.piattaformaUnitaria.enable-exponential-backoff=true
resilience4j.retry.instances.piattaformaUnitaria.exponential-backoff-multiplier=2
```

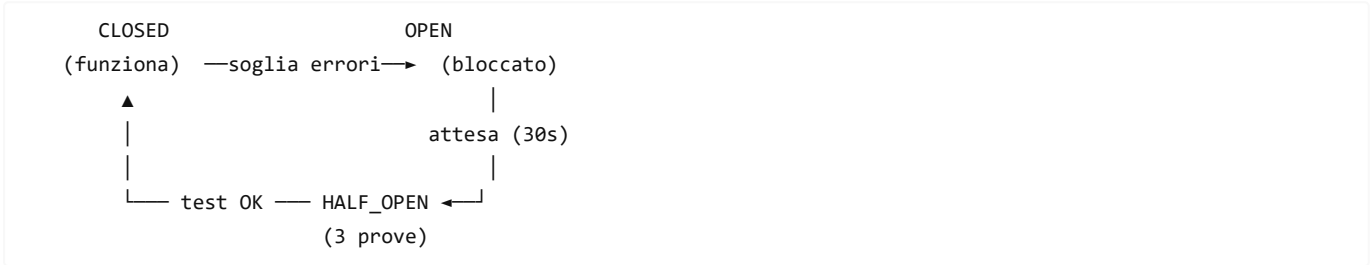
Monitoraggio

Resilience4j espone le metriche tramite Spring Boot Actuator. Gli endpoint disponibili sono:

- `/actuator/circuitbreakers` : Stato dei circuit breaker
- `/actuator/retries` : Statistiche dei retry
- `/actuator/circuitbreakers/events` : Eventi di transizione degli stati

Circuit Breaker (`piattaformaUnitaria`)

Il circuit breaker protegge il sistema da chiamate continue a un servizio non disponibile.



Parametri di default (profilo base):

- Finestra: ultime **10 chiamate** (`COUNT_BASED`)
- Soglia di apertura: **50%** di fallimenti
- Attesa in stato OPEN: **30 secondi**
- Chiamate in HALF_OPEN: **3**
- Eccezioni registrate: `PiattaformaCommunicationException` , `ResourceAccessException`
- Eccezioni ignorate: `PiattaformaAuthenticationException` (gestita con refresh)

Fallback: Quando il circuit breaker è aperto, viene invocato `forwardSoapRequestFallback()` che lancia una `PiattaformaCommunicationException` con **sempre HTTP 503** (Service Unavailable), indipendentemente dal codice HTTP dell'errore originale. Il `SoapFaultExceptionResolver` la converte in `COMM_ERROR` . Prima del fix del 31 Mar 2026, il fallback propagava il codice HTTP dell'errore originale (es. 404), causando messaggi confusi come "HTTP 404 — circuit breaker aperto".

Retry (`piattaformaUnitaria`)

Il retry riprova automaticamente le chiamate fallite con backoff esponenziale.

Parametri di default:

- Tentativi massimi: **3**
- Attesa iniziale: **1 secondo**
- Moltiplicatore esponenziale: **2x** (1s → 2s → 4s)
- Riprova su: `PiattaformaCommunicationException` , `ResourceAccessException`
- Non riprova su: `PiattaformaAuthenticationException`

15. Ambienti e Profili

Il progetto utilizza attualmente **un solo profilo attivo**: `dev` . I profili `uat` e `prod` sono stati rimossi per semplificare l'ambiente di sviluppo; verranno ricreati quando necessario per il deployment in ambienti superiori.

Tutti i file di configurazione sono in formato `.properties` (la migrazione da `.yaml` è avvenuta contestualmente alla semplificazione dei profili).

Panoramica profili

Profilo	Stato	Piattaforma target	Sicurezza	Logging
dev	Attivo	<code>api.uat.p4pa.pagopa.it</code>	JWT disabilitato (anonymous)	DEBUG
uat	Da creare	<code>api.uat.p4pa.pagopa.it</code>	JWT abilitato	INFO
prod	Da creare	URL da env var	JWT abilitato + segreti da env var	WARN

(base)	Sempre attivo	api.uat.p4pa.pagopa.it	JWT su /ws/**	INFO
--------	---------------	------------------------	---------------	------

Profilo `dev` (`application-dev.properties`)

Quando usarlo: Sviluppo che richiede connessione reale all'ambiente UAT di pagoPA.

Caratteristiche:

- Punta all'ambiente UAT reale (`api.uat.p4pa.pagopa.it`)
- **Nessuna credenziale OAuth2 globale** — le credenziali sono per-ente nella tabella `mygov_ente_config_pu`
- **Sicurezza JWT disabilitata** — i SIL non inviano JWT; l'autenticazione avviene tramite `codIpaEnte` + `password` nel body SOAP
- Configurazione SpringLine2 security: `jwt.enabled=false` , `anonymous` per `/**`
- Logging DEBUG per il codice del middleware, Spring WS e RestTemplate
- Actuator health con dettagli sempre visibili (`show-details=always`)
- Resilience4j con parametri rilassati per non ostacolare il debugging (soglia 80%, attesa 10s)
- Database PostgreSQL locale (`localhost:5432/mypay_local_copy`), con supporto override tramite variabili d'ambiente `DB_PA_URL` , `DB_PA_USERNAME` , `DB_PA_PASSWORD`

Profilo base (senza suffisso — `application.properties`)

È la configurazione condivisa, sempre caricata. Il profilo `dev` sovrascrive solo le proprietà che gli servono.

Configurazioni principali:

- Nessuna configurazione JPA/Hibernate: il layer dati usa JDBC + Jdbi
- Piattaforma target: `api.uat.p4pa.pagopa.it` (default UAT)
- Resilience4j: parametri standard (finestra 10 chiamate, soglia 50%, attesa OPEN 30s)
- Actuator: endpoints `health`, `info`, `metrics`, `circuitbreakers`, `retries`
- SpringLine2 security: JWT richiesto su `/ws/**` , anonymous su Swagger e Actuator health
- La configurazione del DataSource **non è nel profilo base** — ogni profilo dichiara la propria connessione

Profili `uat` e `prod` (da creare in futuro)

I profili `uat` e `prod` verranno creati come file `application-uat.properties` e `application-prod.properties` quando necessario per il deployment.

Variabili d'ambiente previste per il profilo `prod` :

Variabile	Descrizione
<code>PIATTAFORMA_BASE_URL</code>	URL base della Piattaforma Unitaria
<code>SPL_JWT_CYPHER_SECRET</code>	Segreto per cifratura JWT SpringLine2

16. Test

Strategia di testing attuale

Il testing del middleware avviene esclusivamente tramite la **collection Postman E2E**:

- **Documento di riferimento:** `docs/procedures/GUIDA_TEST_POSTMAN_END_TO_END.md`
- **Collection:** `requests/MyPay-Middleware-Dev.postman_collection.json`
- **Approccio:** test end-to-end con l'applicazione avviata in profilo `dev` , PostgreSQL attivo e connettività verso la PU UAT reale

Scenari di test coperti dalla collection Postman

Scenario	Cosa verifica
Health check GET /actuator/health	Stato del sistema (DB, PU, token cache)
Richiesta SOAP verso PU (ente con attivo=true)	Flusso OAuth2 per-ente, inoltro alla PU, risposta al SIL
Richiesta SOAP in modalità LEGACY (ente senza config PU)	Forward diretto al backend senza OAuth2
Richiesta SOAP con ente non censito	SOAP Fault ENTE_NON_AUTORIZZATO
Richiesta SOAP con path non riconosciuto	SOAP Fault PATH_NON_RICONOSCIUTO
Verifica token in cache	GET /actuator/health/OAuthToken
Verifica metriche	GET /actuator/metrics/middleware.enti.totali

Build state attuale

```
mvn compile → BUILD SUCCESS (55 source files, 0 errori)
```

Collection Postman

La collection `requests/MyPay-Middleware-Dev.postman_collection.json` contiene:

- **48 richieste** organizzate in **12 cartelle**
- Ogni endpoint SOAP ha almeno una richiesta di test
- Include scenari di errore (ente non censito, path non riconosciuto)
- Health check e metriche Actuator

Variabili della collection

Tutte le variabili sono configurabili direttamente in Postman (tab "Variables" della collection).

Variabile	Valore di default	Descrizione
baseUr1	http://localhost:8080	URL base del middleware
silCodIpaEnte	SELC_99999000013	Codice IPA dell'ente SIL di test
silIdentificativoDominio	99999000013	Codice fiscale dell'ente SIL di test (usato nei body CCP25/MyPay)
silPassword	TEST_PASSWORD	Password SIL di test (campo <password> nel body SOAP)

Per eseguire i test su un ente diverso, è sufficiente modificare queste tre variabili nella collection senza toccare le singole richieste.
Per i dettagli sui test E2E, vedere `docs/procedures/GUIDA_TEST_POSTMAN_END_TO_END.md`.

17. Come avviare il progetto

Pre-requisiti

- Java 17 (C:\Program Files\Java\jdk-17)
- Maven 3.9.9 (C:\Program Files\apache-maven\apache-maven-3.9.9)
- Ambiente Windows con WSL (per gli strumenti di sviluppo)

Compilazione

```
cmd.exe /c "set JAVA_HOME=C:\Program Files\Java\jdk-17&& mvn compile -pl mypay.mypaycore-springboot -am -Denforcer.skip=true"
```

Esecuzione dei test

I test vengono eseguiti tramite la collection Postman E2E. Vedere sezione 16 per i dettagli.

Build completa (tutti i moduli)

```
cmd.exe /c "set JAVA_HOME=C:\Program Files\Java\jdk-17&& mvn clean install -Denforcer.skip=true"
```

Avvio in modalità dev (connessione a PU UAT reale)

```
cmd.exe /c "set JAVA_HOME=C:\Program Files\Java\jdk-17&& set PIATTAFORMA_CLIENT_SECRET=<client-secret>&& mvn spring-boot:run -f mypay.mypaycore-springboot/pom.xml -Denforcer.skip=true -Dspring-boot.run.profiles=dev"
```

NOTA: Sostituire `<client-secret>` con il valore reale. Le credenziali sono nel file `.env` (gitignored). Richiede PostgreSQL attivo su `localhost:5432/mypay_local_copy`.

Configurazione IntelliJ IDEA

1. **Run** → **Edit Configurations** → + → **Maven**

2. Configurare:

- **Name:** `MyPayCore - dev`
- **Command:** `spring-boot:run -f mypay.mypaycore-springboot/pom.xml -Denforcer.skip=true -Dspring-boot.run.profiles=dev`
- **Working directory:** root del progetto (`mypay.mypaycore`)

Note sul flag `-Denforcer.skip=true`

Il POM padre corporativo (`it.ariaspa:cm:1.0.0`) ha un plugin enforcer che verifica che il sistema operativo sia Unix. Poiché lo sviluppo avviene su Windows, questo flag è necessario per bypassare il controllo. **Non usarlo mai in ambienti CI/CD che girano su Linux.**

18. Come testare il flusso completo

17.1 Test con profilo `dev` (PU UAT reale) — TEST END-TO-END

Con l'applicazione avviata in profilo `dev` su `http://localhost:8080` :

Pre-requisiti

1. PostgreSQL attivo su `localhost:5432/mypay_local_copy` (user: admin, password: admin)
2. Variabile d'ambiente `PIATTAFORMA_CLIENT_SECRET` impostata con il client-secret OAuth2 reale
3. Connettività di rete verso `api.uat.p4pa.pagopa.it`

Avvio

```
cmd.exe /c "set JAVA_HOME=C:\Program Files\Java\jdk-17&& set PIATTAFORMA_CLIENT_SECRET=<client-secret>&& mvn spring-boot:run -f mypay.mypaycore-springboot/pom.xml -Denforcer.skip=true -Dspring-boot.run.profiles=dev"
```

Passo 1 — Health check

```
GET http://localhost:8080/actuator/health
```

Risposta attesa (componenti chiave):

- `db` : UP — PostgreSQL connesso
- `piattaformaUnitaria` : UP — PU UAT raggiungibile

- `OAuthToken` : DOWN al primo avvio (normale — il token viene acquisito alla prima richiesta SOAP)

Passo 2 — Chiamata SOAP reale verso PU

Importare in **Postman**: `requests/MyPay-Middleware-Dev.postman_collection.json`

Oppure chiamata diretta:

```
POST http://localhost:8080/ws/pivot/PagamentiTelematiciPagatiRiconciliati
Content-Type: text/xml; charset=UTF-8
```

Oppure chiamata diretta:

```
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ppt="http://www.regione.veneto.it/pagamenti/pivot/ente/ppthead"
  xmlns:ente="http://www.regione.veneto.it/pagamenti/pivot/ente/">
  <soapenv:Header>
    <ppt:intestazionePPT>
      <codIpaEnte>SELC_99999000013</codIpaEnte>
    </ppt:intestazionePPT>
  </soapenv:Header>
  <soapenv:Body>
    <ente:pivotSILAutorizzaImportFlussoTesoreria>
      <password>BERGAMO</password>
      <tipoFlusso>0</tipoFlusso>
    </ente:pivotSILAutorizzaImportFlussoTesoreria>
  </soapenv:Body>
</soapenv:Envelope>
```

Risposta attesa dalla PU reale (HTTP 200):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:pivotSILAutorizzaImportFlussoTesoreriaRisposta
      xmlns:ns3="http://www.regione.veneto.it/pagamenti/pivot/ente/">
      <uploadUrl>https://api.uat.p4pa.pagopa.it/pu/fileshare/organization/...</uploadUrl>
      <authorizationToken>AUTHORIZATIONTOKEN</authorizationToken>
      <requestToken>XXXX</requestToken>
      <importPath>/IMPORTPATH</importPath>
    </ns3:pivotSILAutorizzaImportFlussoTesoreriaRisposta>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Flusso interno reale (post Fase 8 + refactoring multi-ente):

```
Postman → Endpoint concreto (es. ReconciliationEndpoint, Spring WS)
  → processRequest() (ereditato da AbstractSoapProxyEndpoint)
  → estrae Envelope SOAP completo (Header + Body)
  → extractEnteIdentifier(soapEnvelope)
    → cerca <codIpaEnte> o <identificativoDominio>
    → se identificativoDominio → enteCacheService.findByCodiceFiscale()
  → RoutingDecisionService.decide(codIpaEnte, requestPath)
    → EnteCacheService.findByCodIpaEnte(codIpaEnte)
      → cache duale (cacheByCodIpa + cacheByCodiceFiscale)
```



```

→ PiattaformaUnitariaClient.forwardSoapRequest(platformPath, xml, enteCompleto)
  → OAuthTokenService.getAccessToken(codIpaEnte, clientId, clientSecret)
    → POST api.uat.p4pa.pagopa.it/pu/auth/oauth/token?client_id=...&...
    ← Token OAuth2 reale (validità ~4 ore, cache per-ente)
  → POST api.uat.p4pa.pagopa.it/pu/sil/soap/...
    (con Authorization: Bearer <token-reale>)
  ← Risposta SOAP reale dalla PU
→ estrae body dalla risposta PU
→ risposta SOAP a Postman

```

Passo 3 — Verifica token in cache

Dopo la prima richiesta SOAP, verificare che il token sia stato acquisito:

```
GET http://localhost:8080/actuator/health/OAuthToken
```

Risposta attesa: {"status":"UP","details":{"stato":"Token OAuth2 in cache valido"}}

19. Cosa NON è ancora implementato

Questa sezione è fondamentale per chi prende in carico il progetto: elenca esplicitamente le funzionalità **intenzionalmente escluse** dalle fasi completate finora.

Funzionalità	Stato	Note
Schema e tabelle del database PostgreSQL	✓ Implementato	Tabelle mygov_ente_config_pu e mygov_mw_transaction_log, DAO Jdbi, cache TTL per-ente
Routing per modalità (PU vs legacy)	✓ Implementato	RoutingDecisionService — decide dove instradare in base a path + presenza config PU
Log transazionale, audit, metriche	✓ Implementato	TransactionLoggingService, MiddlewareMetricsService, EnteConfigHealthIndicator
Credenziali OAuth2 per-ente	✓ Implementato	mygov_ente_config_pu — ogni ente ha il proprio client_id e client_secret
Proxy upload flusso import	✓ Implementato	UploadProxyCacheService, UploadFlussoController, UploadForwardingClient, UpdateFilePuService, post-processing paaSILAutorizzaImportFlusso e pivotSILAutorizzaImportFlusso, cache key condizionale (requestToken per PU, authorizationToken per LEGACY), campo endpointOrigine
Trasformazione tracciati upload PU 1_4 / 1_5 verso 2.0	Non implementata	Le versioni 1_4 e 1_5 vengono riconosciute ma attualmente rifiutate in attesa della conversione del tracciato
Logica di business (riconciliazione, tesoreria)	Non implementata	Gli endpoint fanno solo forwarding del payload
Trasformazione payload SOAP	Non implementata	Il payload viene inoltrato così com'è senza modifiche (salvo il post-processing upload)
Validazione business dei dati in ingresso	Non implementata	Spring WS valida solo il namespace/localPart

Endpoint SOAP aggiuntivi	✓ Implementato	40 operazioni su 10 endpoint — AbstractSoapProxyEndpoint + 4 MyPay PA + 5 MyPay FESP + 1 MyPivot, identificazione ente duale (codIpaEnte + identificativoDominio), cache duale
Contract-first (WSDL/XSD)	Non implementato	Approccio contract-last corrente
Messaggistica asincrona (JMS/ActiveMQ)	Non implementata	springline2-jms commentato nel pom
Multi-tenancy (più enti su stessa istanza)	✓ Implementato	Cache token per-ente, credenziali OAuth2 per-ente
Rate limiting per SIL	Non implementato	Potrebbe essere necessario con più enti

20. Prossimi passi — Fasi Future

Nota: Le fasi elencate qui sono allineate con `docs/guidelines/Plan.md`, che è il documento di riferimento autoritativo per il piano di implementazione.

Riepilogo funzionalità implementate

- ✓ Fondazioni: pulizia demo, struttura middleware, OAuth2
- ✓ Resilienza, gestione errori, health check
- ✓ Persistenza PostgreSQL: DataSource HikariCP + Jdbi
- ✓ Semplificazione configurazione: eliminazione profili, YAML→Properties
- ✓ Registro path-prefix, configurazione backend, ProxyForwardingClient
- ✓ Schema DB: tabelle `mygov_ente_config_pu` e `mygov_mw_transaction_log`, DAO Jdbi, cache TTL
- ✓ Logica di routing: `RoutingDecisionService`, eccezioni, refactoring endpoint
- ✓ Endpoint SOAP completi: 40 operazioni su 10 endpoint, `AbstractSoapProxyEndpoint`, identificazione ente duale, cache duale
- ✓ Log transazionale, metriche Micrometer, health check enti configurati
- ✓ Refactoring multi-ente: credenziali OAuth2 per-ente, schema `mygov_ente_config_pu`
- ✓ Proxy upload flusso import: `UploadProxyCacheService`, `UploadFlussoController`, `UploadForwardingClient`, `UpdateFilePuService`, validazione versione file verso PU, post-processing `paaSILAutorizzaImportFlusso`

Funzionalità da implementare

- Trasformazione tracciati upload PU `1_4` / `1_5` verso `2.0`
- Profili `uat` e `prod` per deployment

21. Agenti OpenCode

Il progetto utilizza **OpenCode** come strumento di sviluppo assistito da AI. Gli agenti personalizzati sono definiti in `.opencode/agents/` e le regole globali per tutti gli agenti si trovano in `AGENTS.md` nella root del repository.

Agenti disponibili

Agente	File	Quando usarlo
@expert	<code>.opencode/agents/expert.md</code>	Decisioni architetturali, code review, debugging complesso
@planner	<code>.opencode/agents/planner.md</code>	Pianificare nuove fasi, aggiornare docs/, allineare Plan.md
@tester	<code>.opencode/agents/tester.md</code>	Scrivere test unitari Java, test integrazione, gestire Postman

@orchestrator	.opencode/agents/orchestrator.md	Gestire ecosistema AI: agenti, skill, comandi
---------------	----------------------------------	---

Come invocare un agente

In OpenCode, gli agenti si invocano con `@nome-agente` nella chat. Ad esempio:

```
@planner aggiorna Plan.md con le modifiche della Fase 2
@planner pianifica la Fase 3 – logica di business
```

Regole globali (AGENTS.md)

Il file `AGENTS.md` nella root del progetto definisce:

- Struttura del repository e contesto del progetto
- Vincoli tecnici obbligatori (profili, datasource, sicurezza XXE)
- Comandi di build e test per ambiente Windows/WSL
- Convenzioni di documentazione (italiano, versioning semantico)

Appendice — Glossario

Termine	Significato
SIL	Sistemi Informativi Locali — i sistemi degli enti pubblici (comuni, province, ecc.) che inviano richieste al middleware
Piattaforma Unitaria	Sistema di pagoPA che gestisce i pagamenti elettronici degli enti pubblici
pagoPA	Piattaforma nazionale per i pagamenti verso la Pubblica Amministrazione
OAuth2 Client Credentials	Flusso OAuth2 machine-to-machine senza interazione utente: il client si autentica con <code>client_id</code> e <code>client_secret</code> e ottiene un token JWT
SOAP	Simple Object Access Protocol — protocollo per lo scambio di messaggi XML tra sistemi
Spring WS	Modulo Spring per la creazione di endpoint SOAP server-side
SpringLine2	Framework proprietario ARIA S.p.A. che estende Spring Boot con componenti per sicurezza, logging e configurazione
Circuit Breaker	Pattern di resilienza: interrompe le chiamate a un servizio non disponibile per evitare cascate di errori
Resilience4j	Libreria Java per i pattern di resilienza (Circuit Breaker, Retry, Rate Limiter, ecc.)
Actuator	Modulo Spring Boot che espone endpoint HTTP per il monitoraggio dell'applicazione
XXE	XML External Entity — categoria di attacchi che sfruttano il parser XML per leggere file locali o fare richieste di rete
Contract-last	Approccio SOAP in cui il contratto (WSDL) viene generato dal codice; opposto di contract-first
Contract-first	Approccio SOAP in cui il codice viene generato dal contratto (WSDL/XSD)
codIpaEnte	Codice IPA dell'ente pubblico — identificativo univoco dell'ente nel sistema
codiceFiscaleEnte	Codice fiscale dell'ente — chiave alternativa usata come <code>identificativoDominio</code> nelle richieste SOAP degli endpoint FESP
identificativoDominio	Tag XML presente nelle richieste SOAP FESP che contiene il codice fiscale dell'ente — viene risolto in <code>codIpaEnte</code> tramite la cache duale

AbstractSoapProxyEndpoint	Classe base astratta che implementa il pattern proxy SOAP trasparente con identificazione ente duale, routing, logging e metriche — tutte le 10 sottoclassi endpoint la estendono
tipoFlusso	Tipo di flusso di tesoreria (o = Ordinario, F = Finanziario)
mygov_ente	Tabella nel DB condiviso (con mypay/mypivot) che contiene l'anagrafica degli enti pubblici — non creata dal middleware
mygov_ente_config_pu	Tabella del middleware che contiene le credenziali OAuth2 per-ente per la Piattaforma Unitaria — relazione 1:1 con mygov_ente
EnteCompleto	Aggregato Java che unisce Ente + EnteConfigPu — usato da EnteCacheService e RoutingDecisionService
EnteCacheService	Servizio che mantiene in cache (TTL configurabile) il risultato di mygov_ente LEFT JOIN mygov_ente_config_pu per evitare query DB a ogni richiesta SOAP
UploadProxyEntry	DTO immutabile che rappresenta una voce nella cache del proxy upload: contiene l'uploadUrl originale del backend, l'authorizationTokenSil (requestToken per PU, token reale per LEGACY), il requestToken, la modalità di routing, il codice IPA, il timestamp di creazione e l'endpointOrigine (MYPAY o MYPIVOT)
UploadProxyCacheService	Cache TTL in-memory one-shot che associa la chiave di cache all'UploadProxyEntry corrispondente. La chiave è condizionale: requestToken per routing PU (la PU non restituisce un authorizationToken univoco), authorizationToken per routing LEGACY. Ogni entry è consumabile una sola volta
UploadFlussoController	Controller REST (POST /api/upload/flusso) che riceve il file dal SIL, recupera l'entry dalla cache, valida la versione nei casi MyPay+PU (saltata per MyPivot) e lo inoltra al backend originale tramite UploadForwardingClient
UpdateFilePuService	Servizio che estrae la versione dal nome file e decide se l'upload verso PU può essere inoltrato o deve essere bloccato con errore applicativo
SupportedFileVersion	Enum che rappresenta le versioni file riconosciute, mantenendo il mapping tra formato canonico (1.4) e formato filename (1_4)
UploadForwardingClient	Client HTTP che esegue il POST multipart del file verso il backend (legacy o PU) aggiungendo l'Authorization: Bearer solo in modalità PIATTAFORMA_UNITARIA
authorizationToken	Campo nella risposta SOAP di autorizzazione import flusso. Per routing LEGACY contiene il token univoco reale del backend; per routing PU viene sostituito con requestToken (univoco) perché la PU restituisce un valore costante. Usato dal SIL come parametro nella chiamata di upload e come chiave di lookup nella cache
requestToken	Token univoco della richiesta (UUID generato dal backend). Usato come chiave di cache per le richieste instradate verso PU, in sostituzione dell'authorizationToken non univoco
endpointOrigine	Campo di UploadProxyEntry che indica l'endpoint SOAP di origine della richiesta (BackendDestinatario.MYPAY O BackendDestinatario.MYPIVOT). Usato da UploadFlussoController per decidere se applicare la verifica della versione del file (solo per MyPay)